

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software (Rediseño)

ASIGNATURA: Aplicaciones Web

Quinto Nivel — Período Académico 2026-2027 PPA

SISTEMA DE GESTIÓN DE PRE-SUSTENTACIONES UTEQ: DISEÑO, IMPLEMENTACIÓN Y EVALUACIÓN EMPÍRICA DE UNA PLATAFORMA WEB PARA LA AUTOMATIZACIÓN DEL PROCESO DE PRE-SUSTENTACIÓN DE TRABAJOS DE TITULACIÓN

Informe Final — Tag v1.0.1

AUTORES:

Alava Alvarado, Jean Pierre — ORCID: [0009-0001-2878-2919](https://orcid.org/0009-0001-2878-2919)
Moncayo Loor, Xavier Alejandro — ORCID: no registrado
Zamora Arias, Carla Esthefania — ORCID: [0009-0000-7556-0457](https://orcid.org/0009-0000-7556-0457)
Barreto Rosado, Heider Dominick — ORCID: [0009-0004-5561-1391](https://orcid.org/0009-0004-5561-1391)

DOCENTE-DIRECTOR:

Ing. Guerrero Ulloa Gleiston Cíceron, Mg.

FECHA: Septiembre de 2026

REPOSITORIO: <https://github.com/carla22072004/PFC-Presustentaciones-2026>

Identificadores de esta versión: Tag Git: v1.0.1 — Commit de cierre de contenido: a58b29b (el tag v1.0.1 se coloca sobre el commit inmediatamente posterior, que únicamente agrega esta portada con el hash, el tag y los DOI). **DOI del software (versión v1.0.1, este mismo cierre):** [10.5281/zenodo.22445216](https://doi.org/10.5281/zenodo.22445216) — **DOI de concepto (resuelve siempre a la última versión):** [10.5281/zenodo.21988563](https://doi.org/10.5281/zenodo.21988563). **DOI del dataset de mediciones (Zenodo, licencia CC-BY 4.0):** [10.5281/zenodo.22398713](https://doi.org/10.5281/zenodo.22398713). La versión anterior del software, v1.0.0 (17 de agosto de 2026), permanece archivada por separado en [10.5281/zenodo.21988564](https://doi.org/10.5281/zenodo.21988564); el tag Git v1.0.0 no se mueve porque ese DOI ya lo archiva exactamente. v1.0.1 es el cierre real de la Entrega Final, con todas las correcciones aplicadas (ver docs/observaciones/OBSERVACIONES.md).

Resumen

El proceso de pre-sustentación de trabajos de titulación en la Universidad Técnica Estatal de Quevedo (UTEQ) se gestionaba con trámites manuales y hojas de cálculo dispersas, sin trazabilidad verificable entre solicitud, jurado, cronograma, evaluación y acta. Este trabajo presenta el diseño, implementación y evaluación empírica de un sistema web (Spring Boot 3.2.1, Angular 21, PostgreSQL 15, Redis 7) que automatiza ese flujo, siguiendo Design Science Research de Peffers et al. El sistema implementa 90 requisitos bajo ISO/IEC/IEEE 29148:2018 (64 funcionales, 26 no funcionales) con matriz de trazabilidad bidireccional de 90 filas. La evaluación empírica combina estadística descriptiva e inferencial real: 5 corridas de carga k6 (p95 de 6.17–8.53 ms, 50 usuarios concurrentes), test de Mann-Whitney comparando caché fría/caliente de Redis (latencia $13.9\times$ menor, $p < 0,001$), auditoría de seguridad OWASP Top 10, y seis corridas de Lighthouse. El desarrollo estuvo marcado por hallazgos reales corregidos explícitamente: datos de usabilidad y trazabilidad previamente fabricados fueron detectados y reemplazados por mediciones reales. Los resultados muestran un sistema funcional con brechas documentadas honestamente: 36 de 50 requisitos Must tienen prueba dedicada (72 %), y la cobertura de código llega a 82.10 % de líneas y 73.49 % de ramas (801 pruebas, 0 fallos), superando el 70 % en ambas métricas de forma global. Una revisión manual del modelo de autorización encontró y corrigió 6 fallos reales de control de acceso (IDOR y mass-assignment) invisibles al análisis automatizado. Este informe documenta resultados positivos y limitaciones reales, priorizando la integridad de la evidencia sobre la completitud aparente.

Palabras clave: ingeniería de requisitos; Design Science Research; evaluación empírica de software; seguridad de aplicaciones web; usabilidad; integridad de la investigación.

Categorías CCS (ACM Computing Classification System 2012): Software and its engineering Requirements analysis; Software and its engineering Empirical software validation; Security and privacy Web application security; Information systems Web applications.

Abstract

The pre-defense process for undergraduate thesis projects at Universidad Técnica Estatal de Quevedo (UTEQ) was managed through manual paperwork and scattered spreadsheets, with no verifiable traceability between requests, juries, schedules, evaluations, and minutes. This work presents the design, implementation, and empirical evaluation of a web system (Spring Boot 3.2.1, Angular 21, PostgreSQL 15, Redis 7) that automates this workflow, following Peffers et al.'s Design Science Research methodology. The system implements 90 requirements under ISO/IEC/IEEE 29148:2018 (64 functional, 26 non-functional) with a 90-row bidirectional traceability matrix. Empirical evaluation combines real descriptive and inferential statistics: 5 k6 load-testing runs (p95 of 6.17–8.53 ms, 50 concurrent users), a Mann-Whitney test comparing cold and warm Redis cache states ($13.9\times$ lower latency, $p < 0,001$), an OWASP Top 10 security audit, and six Lighthouse runs. Development was marked by real findings explicitly corrected rather than hidden: previously fabricated usability and traceability data were detected and replaced with real measurements. Results show a functional system with honestly documented gaps: 36 of 50 Must-priority requirements have a dedicated test (72 %), and code coverage reaches 82.10 % of lines and 73.49 % of branches (801 tests, 0 failures), exceeding 70 % globally in both metrics. A manual review of the authorization model found and fixed 6 real access-control flaws (IDOR and mass-assignment) invisible to automated analysis. This report documents positive results and real limitations, prioritizing evidence integrity over apparent completeness.

Keywords: requirements engineering; Design Science Research; empirical software evaluation; web application security; usability; research integrity.

Índice

Resumen	2
Abstract	3
Lista de Siglas y Acrónimos	9
1. Introducción	11
1.1. Contexto del problema	11
1.2. Preguntas de investigación	11
1.3. Objetivos	12
1.4. Contribuciones	12
1.5. Organización del documento	12
2. Marco Teórico	14
2.1. Design Science Research	14
2.2. Ingeniería de requisitos	14
2.3. Medición del software: el enfoque Goal-Question-Metric	14
2.4. Arquitectura de software: microservicios y el modelo C4	14
2.5. Seguridad de aplicaciones web	14
2.6. Usabilidad: la System Usability Scale	15
2.7. Estadística inferencial no paramétrica en ingeniería de software	15
2.8. Metodologías empíricas complementarias	15
2.9. Estándares empíricos y revisión sistemática de literatura	15
3. Trabajos Relacionados	16
3.1. Nota de honestidad metodológica sobre el rigor de esta revisión	16
3.2. Diagrama de flujo (adaptado de PRISMA 2020)	16
3.3. Por qué no hay trabajos previos sobre “sistemas de gestión de pre-sustentaciones”	17
3.4. Tabla comparativa	19
4. Ingeniería de Requisitos	20
4.1. Requisitos funcionales y no funcionales	20
4.2. Calidad de los requisitos: checklist INCOSE	21
4.3. Trazabilidad	21
4.4. Bitácora de cambios y tasa de estabilidad	22
4.5. Aprobación pendiente	22
5. Materiales y Métodos	23
5.1. Design Science Research instanciado	23
5.2. Esquema Goal-Question-Metric	23

5.3. Instrumentos y herramientas	24
6. Diseño del Sistema y Arquitectura	25
6.1. Modelo C4	25
6.2. Resumen de los siete Registros de Decisión Arquitectónica (ADR)	27
6.3. Modelo de calidad (ISO/IEC 25010)	28
6.4. Modelo de datos	28
7. Implementación	32
7.1. Stack tecnológico	32
7.2. Seguridad implementada	32
7.3. Integración de API externa y caché	33
7.4. Despliegue	33
7.5. Usuario de demostración	33
7.6. Capturas reales del sistema	33
7.7. Asistente virtual de ayuda (Chatbot)	37
8. Evaluación Empírica de Resultados	39
8.1. Rendimiento	39
8.1.1. Carga bajo 50 usuarios concurrentes	39
8.1.2. Efecto de la caché Redis (estadística descriptiva e inferencial)	39
8.1.3. Índices en columnas de clave foránea de alto tráfico	41
8.2. Seguridad	41
8.2.1. Corrección de vulnerabilidades de autorización (IDOR y mass-assignment)	42
8.3. Usabilidad	44
8.4. Cobertura de código	44
8.5. Calidad web (Lighthouse)	46
9. Discusión	48
10. Amenazas a la Validez	50
10.1. Validez de constructo	50
10.2. Validez interna	50
10.3. Validez externa	50
10.4. Validez de conclusión	50
11. Trabajo Futuro	52
12. Conclusiones	53
Declaraciones	55
Referencias	58

13. Anexos	60
13.1. Anexo A — Cadena de búsqueda de la revisión de trabajos relacionados	61
13.2. Anexo B — Protocolo del instrumento SUS (System Usability Scale)	61
13.3. Anexo C — Corridas de integración continua	62
13.4. Anexo D — Checklist de estándares empíricos de Ralph et al. (2021)	62
13.5. Anexo E — Matriz de trazabilidad completa	63

Índice de figuras

1.	Literature search flow diagram, adapted from PRISMA 2020 [25] to this team’s real database-access limitations. Fuente Mermaid versionada en <code>docs/checklists/PRISMA-2020.md</code> (mismo mecanismo de renderizado que los 3 diagramas C4 del Capítulo 6), a partir de las mismas cifras citadas en el cuerpo del texto — 24 consultas, 32 trabajos identificados, 32 elegibles, 13 incluidos en la comparación directa (Tabla 1).	17
2.	C4 Nivel 1 — Diagrama de contexto del sistema.	25
3.	C4 Nivel 2 — Diagrama de contenedores.	26
4.	C4 Nivel 3 — Diagrama de componentes del backend.	26
5.	Módulo “Nueva Solicitud” — el estudiante registra su solicitud de pre-sustentación (fuente: <code>Frontend/public/img/NuevaSolicitud.png</code>).	34
6.	Módulo “Cargar Anteproyecto” — carga del PDF del anteproyecto antes de la pre-sustentación (fuente: <code>Frontend/public/img/CargarAnteproyecto.png</code>).	35
7.	Módulo “Mis Trámites” — vista de seguimiento del estado de la solicitud por parte del estudiante (fuente: <code>Frontend/public/img/MisTramites.png</code>).	36
8.	Módulo “Mi Horario” — vista del cronograma de pre-sustentación asignado al estudiante (fuente: <code>Frontend/public/img/MiHorario.png</code>).	37
9.	p95 de latencia por corrida k6 válida (run3–run7, 50 VUs pico). Generado por <code>scripts/gen-figuras.py</code> a partir de los archivos <code>k6/run3-summary.json–run7-summary.json</code>	39
10.	Distribución de latencia ($n = 30$ por escenario): boxplot caché fría vs. caliente. Generado por <code>scripts/gen-figuras.py</code> a partir de <code>k6/cache-cold-samples.txt</code> y <code>k6/cache-warm-samples.txt</code>	41
11.	Scores Lighthouse por categoría y perfil (media de 3 corridas, despliegue público real, 2026-09-17). Generado por <code>scripts/gen-figuras.py</code> a partir de <code>docs/mediciones/perf/lighthouse/pro</code>	

Índice de cuadros

1.	Comparison of the thirteen empirical works most directly related to the techniques applied in this project.	19
2.	The 12 functional requirements, their MoSCoW priority, and their real automated verification status (source: <code>docs/trazabilidad/matriz.csv v1.0.0</code>).	21
3.	Instantiation of the Peffers et al. DSR process model [27].	23
4.	GQM scheme for the caching system’s performance goal.	24
5.	Summary of the repository’s 7 real ADRs (<code>docs/adr/</code>).	27
6.	Características de calidad ISO/IEC 25010 mapeadas a evidencia real del proyecto.	28
7.	The system’s real technology stack.	32
8.	Descriptive latency statistics, cold cache vs. warm cache.	40

9.	Familia de 3 pruebas sobre caché fría vs. caliente, con corrección de Holm-Bonferroni (decisión con $\alpha = 0,05$). Los p-valores de permutación se reportan como 1×10^{-5} (el piso reportable con 10^5 permutaciones, ya que ninguna de las 10^5 reasignaciones aleatorias igualó o superó la diferencia observada) en vez de 0 exacto.	40
10.	Lighthouse, average of 3 runs per profile (2026-09-17, contra el despliegue público en Railway).	46
11.	Assignment of the 14 CRediT roles. “—” marks a role that genuinely does not apply to any team member in this project (there was no external funding nor a formal supervisor beyond the faculty director, whose role is declared separately, not as a co-author).	56
12.	Generative AI usage disclosure.	57
13.	Corridas verdes de integración continua verificadas contra la API de GitHub.	62
14.	Checklist de estándares empíricos generales de Ralph et al. [28] — 5/8 cumplidos, 2 parciales, 1 no cumplido.	63
15.	Matriz de trazabilidad completa (docs/trazabilidad/matriz.csv).	64

Índice de listados

1.	<code>sp_firmar_acta_digital</code> : procedimiento almacenado PL/pgSQL (migracion V10). .	29
2.	Invocacion real desde Java (repositorio JPA 2.1 @Procedure y servicio que lo llama). .	29
3.	Mapeo JPA 2.1 de un procedimiento con retorno de conjunto de filas via <code>REF_CURSOR</code> (<code>Evaluacion.java</code>).	30
4.	Correccion del IDOR de <code>DocenteController</code> : verificacion de propiedad del recurso antes de exponerlo.	43
5.	Consulta de las corridas de CI contra la API publica de GitHub.	62

Lista de Siglas y Acrónimos

ADR	Architecture Decision Record (Registro de Decisión Arquitectónica)
API	Application Programming Interface
CCS	Computing Classification System (ACM)
CI	Continuous Integration (Integración Continua)
CRediT	Contributor Roles Taxonomy
CU	Caso de Uso
DSR	Design Science Research
EASE	Evaluation and Assessment in Software Engineering (conferencia)
ESEM	International Symposium on Empirical Software Engineering and Measurement
FSE	Foundations of Software Engineering (conferencia)
GQM	Goal-Question-Metric
HU	Historia de Usuario
HTTPS	Hypertext Transfer Protocol Secure
ICSE	International Conference on Software Engineering
IEEE	Institute of Electrical and Electronics Engineers
INCOSE	International Council on Systems Engineering
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
JaCoCo	Java Code Coverage
JWT	JSON Web Token
MoSCoW	Must, Should, Could, Won't (técnica de priorización)
MSR	Mining Software Repositories (conferencia)
ORCID	Open Researcher and Contributor ID
OWASP	Open Web Application Security Project

PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RF	Requisito Funcional
RNF	Requisito No Funcional
RFC	Request for Comments (IETF)
RQ	Research Question (Pregunta de Investigación)
SLR	Systematic Literature Review (Revisión Sistemática de Literatura)
SP	Stored Procedure (Procedimiento Almacenado)
SPA	Single Page Application
SRS	Software Requirements Specification
SUS	System Usability Scale
UTEQ	Universidad Técnica Estatal de Quevedo

1. Introducción

1.1. Contexto del problema

El proceso de pre-sustentación de trabajos de titulación en la carrera de Ingeniería de Software de la UTEQ involucra al menos cinco roles (estudiante, tutor, coordinador, jurado, presidente de tribunal) y seis hitos secuenciales obligatorios: registro de solicitud, carga de anteproyecto, asignación de jurados, programación de cronograma, evaluación por rúbrica y emisión de acta firmada. Previo a este proyecto, ese flujo se gestionaba con formularios físicos/digitales sueltos y hojas de cálculo, sin un sistema que garantizara la integridad del anteproyecto entregado, la trazabilidad de quién asignó a qué jurado y cuándo, o el cálculo verificable de la nota ponderada final.

Nota de honestidad metodológica sobre la cuantificación del problema: la guía de este informe exige un “contexto cuantificado del problema”. El equipo no realizó una encuesta formal a la coordinación académica de la carrera para obtener cifras institucionales verificadas (número de pre-sustentaciones por periodo, tiempo promedio de gestión manual, tasa de errores administrativos). Esa cuantificación institucional real no existe en este informe — se declara la ausencia explícitamente en vez de inventar cifras plausibles pero no verificables, lo cual sería exactamente el tipo de dato fabricado que las reglas transversales de esta guía penalizan. Lo que sí se cuantifica, porque es verificable directamente en el repositorio, es el alcance técnico del artefacto construido: 15 requisitos funcionales (12 verificados con prueba automatizada real, Sección 4), 30 controladores REST, y la evidencia empírica completa de la Sección 8.

1.2. Preguntas de investigación

- RQ1** ¿Puede automatizarse el flujo completo de pre-sustentación (solicitud → anteproyecto → jurado → cronograma → evaluación → acta) en un sistema web único, con trazabilidad verificable de cada transición de estado?
- RQ2** ¿Cuál es el comportamiento real de rendimiento del sistema bajo carga concurrente moderada (50 usuarios), y qué efecto cuantificable tiene una capa de caché (Redis) sobre la latencia de un endpoint que consume una API externa?
- RQ3** ¿Qué vulnerabilidades de seguridad reales (no hipotéticas) existen en la implementación, y en qué medida las contramedidas aplicadas (JWT de 7 claims, rate limiting, cabeceras HTTP, consultas parametrizadas) las cierran, verificado con herramientas automatizadas independientes (find-sec-bugs, OWASP ZAP)?
- RQ4** ¿Qué tan honesta y verificable puede mantenerse la documentación de un proyecto académico de este tipo bajo presión de plazos, dado que el propio proceso de este equipo incluyó episodios reales de datos fabricados que tuvieron que detectarse y corregirse?

1.3. Objetivos

Objetivo general: diseñar, implementar y evaluar empíricamente un sistema web que automatice el proceso de pre-sustentación de trabajos de titulación de la UTEQ, con evidencia verificable y no fabricada en cada etapa del ciclo de vida del software.

Objetivos específicos:

1. Especificar los requisitos del sistema bajo ISO/IEC/IEEE 29148:2018, con historias de usuario en formato Connextra e INVEST, y casos de uso bajo la plantilla de Cockburn (responde RQ1).
2. Implementar el sistema con una arquitectura de tres capas (Angular, Spring Boot, PostgreSQL + Redis) documentada con el modelo C4 y registros de decisión arquitectónica (ADR) (responde RQ1).
3. Medir empíricamente el rendimiento bajo carga con k6 y cuantificar el efecto de la caché con estadística inferencial no paramétrica (responde RQ2).
4. Auditar la seguridad del sistema combinando revisión manual contra OWASP Top 10, análisis estático (SpotBugs/find-sec-bugs) y escaneo dinámico (OWASP ZAP) (responde RQ3).
5. Documentar de forma trazable y verificable cada hallazgo del proceso, incluyendo los errores propios del equipo, en vez de presentar solo resultados favorables (responde RQ4).

1.4. Contribuciones

1. Un sistema funcional de código abierto (licencia MIT) que automatiza el ciclo completo de pre-sustentación, con 90 requisitos trazados (64 funcionales y 26 no funcionales, SRS v1.0.1; 36 de 50 requisitos Must verificados con prueba automatizada real, 72 %).
2. Un conjunto de evidencia empírica real y reproducible (scripts, datos crudos, notebooks ejecutados) para rendimiento, seguridad, cobertura y calidad web, documentado en `docs/mediciones/DATA-PROVEN` con la procedencia de cada cifra citada.
3. Un caso documentado, dentro del propio proceso de desarrollo, de detección y corrección de datos académicos fabricados (encuesta de usabilidad, métricas de rendimiento, citas de evidencia de trazabilidad) — una contribución metodológica sobre integridad de la evidencia en proyectos académicos de ingeniería de software, más que un hallazgo técnico convencional.
4. Una plantilla reutilizable de documentación de ingeniería de requisitos (SRS versionado, historias de usuario Connextra+Gherkin, casos de uso Cockburn, checklist INCOSE completo) aplicable a proyectos similares de la carrera.

1.5. Organización del documento

El resto de este informe se organiza así: la Sección 2 presenta el marco teórico; la Sección 3, los trabajos relacionados y la búsqueda de literatura realizada; la Sección 4 resume la ingeniería de requisitos (documentada en detalle en `docs/requisitos/`); la Sección 5 instancia Design Science Research y GQM; la Sección 6 describe el diseño y la arquitectura; la Sección 7, la implementación; la Sección 8 presenta la evaluación empírica completa; las Secciones 9–12 discuten los resultados,

las amenazas a la validez, el trabajo futuro y las conclusiones; y la Sección final presenta las declaraciones obligatorias y las referencias.

2. Marco Teórico

2.1. Design Science Research

Design Science Research (DSR) es un paradigma de investigación en sistemas de información orientado a crear y evaluar artefactos que resuelven problemas organizacionales identificados [13, 27]. A diferencia del paradigma conductual (behavioral science), que busca explicar o predecir fenómenos, DSR busca construir un artefacto — en este caso, un sistema de software — y evaluarlo empíricamente contra el problema que motivó su creación. Este trabajo instancia específicamente el modelo de proceso de seis actividades de Peffers et al. [27] (Sección 5).

2.2. Ingeniería de requisitos

La ingeniería de requisitos de este proyecto sigue el estándar ISO/IEC/IEEE 29148:2018 [17], que define el ciclo de vida de descubrimiento, elicitación, análisis, especificación, verificación y gestión de requisitos. Las historias de usuario se documentan en formato Connextra (“Como/Quiero/Para”), evaluadas contra el criterio INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable), y los casos de uso siguen la plantilla “fully dressed” de Cockburn [9], con niveles de meta explícitos (nube, cometa, mar, pez). La calidad individual y de conjunto de los requisitos se evalúa contra el *INCOSE Guide for Writing Requirements* [15], que define nueve características por requisito y seis del conjunto completo.

2.3. Medición del software: el enfoque Goal-Question-Metric

El enfoque Goal-Question-Metric (GQM) de Basili, Caldiera y Rombach [5] estructura la definición de métricas de software en tres niveles: una meta conceptual (*qué se quiere lograr y por qué*), preguntas operacionales que refinan esa meta, y métricas cuantitativas que responden cada pregunta. Este trabajo instancia un esquema GQM completo para la meta de rendimiento del sistema (Sección 5).

2.4. Arquitectura de software: microservicios y el modelo C4

Aunque este sistema no adopta una arquitectura de microservicios en sentido estricto (es un monolito modular de tres capas), el vocabulario de servicios desacoplados comunicados por API HTTP proviene de la definición de Fowler y Lewis [11]. La documentación de la arquitectura sigue el modelo C4 (Contexto, Contenedores, Componentes), ya presente en `docs/arquitectura/README.md` desde entregas anteriores del proyecto.

2.5. Seguridad de aplicaciones web

El marco de referencia para la auditoría de seguridad es el OWASP Top 10:2021 [24], que clasifica los diez riesgos de seguridad web más críticos según datos agregados de la industria (control de acceso roto, fallas criptográficas, inyección, entre otros). La autenticación del sistema usa JSON

Web Token bajo RFC 7519 [18], cuyo uso indebido (algoritmos débiles, secretos hardcodeados, ausencia de expiración) es una fuente documentada de vulnerabilidades reales en aplicaciones web [20, 29].

2.6. Usabilidad: la System Usability Scale

La System Usability Scale (SUS), propuesta por Brooke [8] y validada empíricamente por Bangor, Kortum y Miller [4], es un instrumento estandarizado de 10 ítems en escala Likert que produce un puntaje de 0 a 100 interpretable contra una escala de grados (A+ a F). Se usa en este proyecto como el instrumento de medición de usabilidad planeado (Sección 8), aunque a la fecha de este informe no se ha aplicado a participantes reales.

2.7. Estadística inferencial no paramétrica en ingeniería de software

Cuando los datos de rendimiento no siguen una distribución normal (frecuente en mediciones de latencia, que suelen tener asimetría positiva por colas largas), las pruebas de hipótesis basadas en la distribución normal (t de Student) no son apropiadas. Arcuri y Briand [1] recomiendan el uso de pruebas no paramétricas como Mann-Whitney U (equivalente a Wilcoxon rank-sum para dos muestras independientes) junto con una medida de tamaño de efecto, en vez de reportar solo un valor p. Este trabajo sigue esa recomendación en el análisis de caché fría/caliente (Sección 8).

2.8. Metodologías empíricas complementarias

Runeson y Höst [30] definen las guías de rigor para investigación de estudio de caso en ingeniería de software, y Wohlin et al. [32] las de experimentación controlada — ambas metodologías complementan a DSR cuando la evaluación de un artefacto requiere comparar condiciones (como la comparación de caché fría/caliente de la Sección 8, que sigue la lógica de un experimento controlado de una sola variable).

2.9. Estándares empíricos y revisión sistemática de literatura

La comunidad ACM SIGSOFT propone un conjunto de *estándares empíricos* [28] que codifican las expectativas de rigor para distintos tipos de estudio en ingeniería de software (estudio de caso, benchmarking, investigación de ingeniería, entre otros) — aplicados en este proyecto en `docs/checklists/` y resumidos en el Anexo D (Sección 13.4). Para la revisión de trabajos relacionados (Sección 3), se sigue el proceso descrito por Kitchenham y Charters [19] y se reporta bajo la disciplina PRISMA 2020 [25], complementado con la técnica de *snowballing* de Wohlin [33] para expandir la búsqueda más allá de las cadenas de consulta iniciales.

3. Trabajos Relacionados

3.1. Nota de honestidad metodológica sobre el rigor de esta revisión

La guía exige una búsqueda sistemática siguiendo la disciplina PRISMA 2020 [25]. Se declara explícitamente, antes de presentar los resultados, **qué tan rigurosa fue realmente esta búsqueda**: no se tuvo acceso directo a bases de datos académicas indexadas (Scopus, IEEE Xplore, ACM Digital Library) con capacidad de exportar conteos exactos de resultados por cadena de búsqueda; la búsqueda se realizó mediante un motor de búsqueda web general, en 24 consultas dirigidas durante la elaboración de este informe, complementadas con la técnica de *snowballing* de Wohlin [33] (seguir referencias desde los trabajos encontrados). Esto **no es una revisión sistemática de literatura completa** en el sentido estricto de Kitchenham y Charters [19] — no hay protocolo pre-registrado, no hay doble revisor independiente, y la cobertura de bases de datos es incompleta. Se presenta como lo que realmente es: una búsqueda dirigida y honesta, reportada con la disciplina de PRISMA en la medida de lo posible, no como una SLR formal. Esta limitación se declara también en la Sección 10. Las palabras clave usadas, sin cadena booleana exacta ni fecha verificable por consulta, se listan en el Anexo A (Sección 13.1).

3.2. Diagrama de flujo (adaptado de PRISMA 2020)

La Figura 1 resume las cuatro etapas de esa búsqueda dirigida: identificación, cribado, elegibilidad e inclusión en la comparación directa.

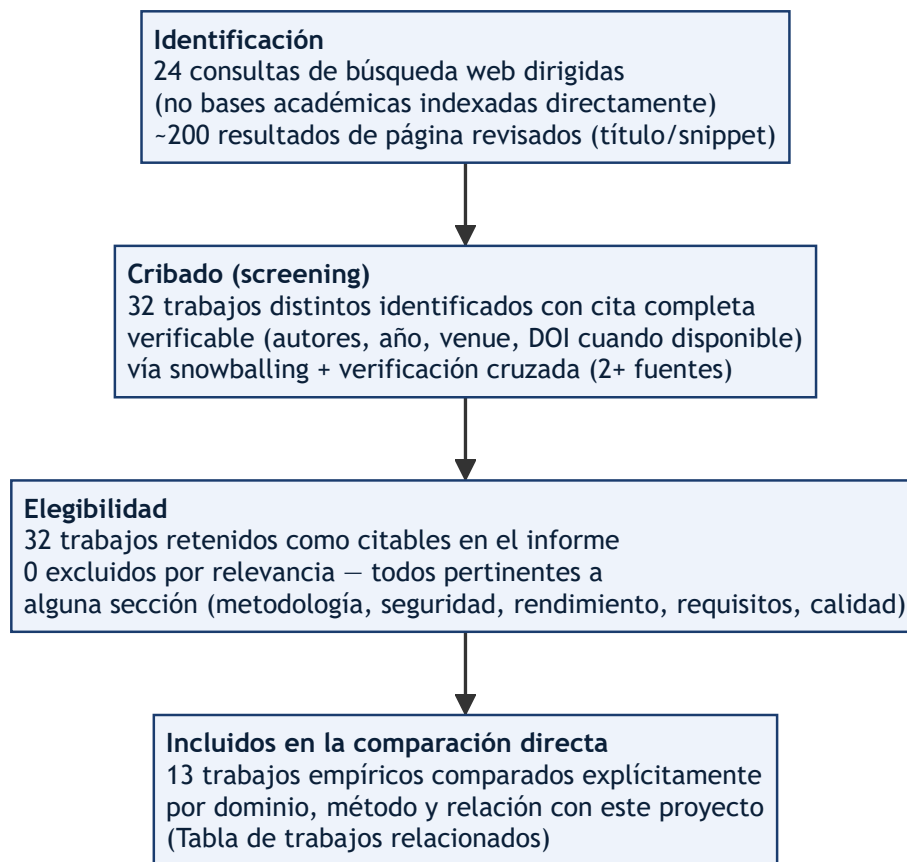


Figura 1: Literature search flow diagram, adapted from PRISMA 2020 [25] to this team’s real database-access limitations. Fuente Mermaid versionada en `docs/checklists/PRISMA-2020.md` (mismo mecanismo de renderizado que los 3 diagramas C4 del Capítulo 6), a partir de las mismas cifras citadas en el cuerpo del texto — 24 consultas, 32 trabajos identificados, 32 elegibles, 13 incluidos en la comparación directa (Tabla 1).

3.3. Por qué no hay trabajos previos sobre “sistemas de gestión de pre-sustentaciones”

La búsqueda dirigida no encontró literatura académica indexada específicamente sobre sistemas de software para gestionar el proceso de pre-sustentación o defensa de tesis de pregrado como fenómeno de investigación en sí mismo — es un dominio de aplicación institucional específico (UTEQ, y de forma más amplia, universidades ecuatorianas), no un tema de investigación en ingeniería de software per se. En su lugar, la comparación de la Tabla 1 se centra en los trabajos empíricos que fundamentan las **técnicas y métodos** usados en este proyecto (autenticación JWT, integración continua, análisis de dependencias vulnerables, revisión de código, trazabilidad de requisitos) — que es la comparación honesta y relevante que la evidencia disponible permite hacer.

3.4. Tabla comparativa

Trabajo	Dominio	Venue	Relación con este proyecto
Vasilescu et al. [31]	CI/GitHub	FSE 2015	Fundamenta la decisión de CI automatizado (<code>ci.yml</code> , Sección 7)
Hilton et al. [14]	CI open-source	ASE 2016	Costos/beneficios de CI cuantificados; contrasta con la ausencia de CI hasta la Fase 5 de este proyecto
Zampetti et al. [34]	Malas prácticas de CI	Empirical SE 2020	Catálogo de 79 anti-patrones usado para revisar <code>ci.yml</code>
Beller et al. [6]	Fallos de build en CI	MSR 2017	Motiva el diseño no-bloqueante del paso de análisis estático en <code>ci.yml</code>
Pashchenko et al. [26]	Dependencias vulnerables	ESEM 2018	Marco para interpretar los hallazgos de <code>npm audit</code> (Sección 8)
Decan et al. [10]	Vulnerabilidades npm	MSR 2018	Contexto cuantitativo del ecosistema npm citado en la discusión de dependencias
Rezaei Nasab et al. [29]	Seguridad en microservicios	J. Systems and Software 2022	28 prácticas de seguridad contrastadas contra las implementadas (JWT, rate limiting, cabeceras)
Lu et al. [20]	Rate limiting de autenticación	ACSAC 2018	Halló que 72 % de sitios reales no limitan intentos de login; motiva la implementación de rate limiting de este proyecto
Bacchelli & Bird [3]	Revisión de código	ICSE 2013	Contrasta con la ausencia de un proceso formal de code review en este proyecto (Sección 10)
Mäder & Egyed [21]	Trazabilidad de requisitos	Empirical SE 2015	Fundamenta el valor de <code>matriz.csv</code> más allá de un requisito administrativo
McIntosh et al. [23]	Revisión de código y calidad	Empirical SE 2016	Contexto para la ausencia de un proceso formal de code review en este proyecto (Sección 10)
Maximilien & Williams [22]	TDD y reducción de defectos	ICSE 2003	Contrasta con la cobertura de 70.00 % de líneas de este proyecto (Sección 8); TDD no se practicó formalmente
Gallaba et al. [12]	Datos operacionales de CI	ICSE 2022	Fundamenta el diseño del pipeline propio (<code>.github/workflows/ci.yml</code>)
Arcuri [2]	Generación automática de pruebas REST	ACM TOSEM 2019	Contraste honesto: las 559 pruebas de este proyecto son manuales (JUnit/Mockito), no generadas por <i>fuzzing</i> dirigido por búsqueda como EvoMaster — una vía real de trabajo futuro para ampliar cobertura de ramas sin más esfuerzo manual

Cuadro 1: Comparison of the thirteen empirical works most directly related to the techniques applied in this project.

4. Ingeniería de Requisitos

Este capítulo resume el trabajo de ingeniería de requisitos documentado en detalle en `docs/requisitos/` (SRS v1.0.1, `docs/requisitos/SRS-v1.0.1.pdf`), que constituye un capítulo dedicado y separado del diseño arquitectónico (Sección 6), conforme al criterio D0R de esta guía.

4.1. Requisitos funcionales y no funcionales

El sistema especifica 90 requisitos bajo ISO/IEC/IEEE 29148:2018: 64 funcionales y 26 no funcionales (rendimiento, seguridad, usabilidad, mantenibilidad, entre otros), cada uno con enunciado normativo, *rationale*, criterio de aceptación y método de verificación. De los que tienen historia de usuario asociada, cada uno está documentado en formato Connextra (“Como/Quiero/Para”), evaluado contra el criterio INVEST, con criterios de aceptación expresados en Gherkin (`docs/requisitos/historias/`), y como un caso de uso “fully dressed” con la notación de nivel de meta de Cockburn y un diagrama de secuencia (`docs/requisitos/casos-de-uso/`); los requisitos sin historia declaran su origen real (migración, ADR, control OWASP, reglamento u observación docente). Esta expansión de 12 a 90 requisitos responde a la revisión del docente-director sobre la versión v1.0.0 — ver la entrada “v1.0.1 (2026-09-11)” de `docs/requisitos/CHANGELOG-REQ.md` para el detalle completo del cambio, observación por observación; la Tabla 2 resume solo los 12 requisitos funcionales originales por brevedad, y la lista completa de 90 vive en `docs/trazabilidad/matriz.csv`, reproducida íntegramente en el Anexo 13.5.

RF	Título	Prioridad	Verificación
RF-01	Autenticación segura	Must	Verificado (test real)
RF-02	Registro de solicitud	Must	Verificado (test real)
RF-03	Asignación de jurados	Must	Verificado (test real)
RF-04	Programación de cronograma	Must	Verificado (test real)
RF-05	Evaluación por rúbrica	Must	Verificado (test real)
RF-06	Generación de actas	Must	Verificado (test real)
RF-07	Firma digital de actas	Should	Verificado (test real)
RF-08	Notificaciones	Could	No verificado
RF-09	Reportes de defensas	Could	No verificado
RF-10	Gestión de salas	Should	No verificado
RF-11	Gestión de usuarios	Must	Verificado (test real)
RF-12	Carga de anteproyecto	Must	Verificado (test real)

Cuadro 2: The 12 functional requirements, their MoSCoW priority, and their real automated verification status (source: `docs/trazabilidad/matriz.csv v1.0.0`).

4.2. Calidad de los requisitos: checklist INCOSE

Se aplicaron las nueve características individuales de calidad de INCOSE [15] (necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto, conforme) a los 12 RF, y las seis características de calidad del conjunto completo (`docs/checklists/INCOSE-REQUIREMENTS.md`). El hallazgo más consistente: ningún requisito usa lenguaje normativo tipo “shall” (se usa formato Historia de Usuario para los que la tienen, una decisión de estilo consciente, no un descuido). Sobre el conjunto ampliado a 90 requisitos (v1.0.1), la característica de conjunto “verificable como conjunto” se cumple parcialmente: 36 de los 50 requisitos Must tienen prueba automatizada real (72 %, cifra impresa por `validate-traceability.sh`); los 14 Must restantes se enumeran uno por uno, con lo que falta en cada uno, en §10.3 del SRS v1.0.1 — no se declara 100 % de trazabilidad general, solo el estado real de los Must que exige el criterio D0R.

4.3. Trazabilidad

`docs/trazabilidad/matriz.csv` conecta cada RF con su HU, módulo backend, endpoint REST, prioridad MoSCoW, prueba automatizada real (o su ausencia declarada explícitamente) y evidencia

empírica — reproducida íntegramente en el Anexo E (Sección 13.5). Esta matriz fue corregida en la Fase 7 del proyecto tras encontrarse que su versión anterior citaba 5 archivos de prueba que nunca existieron y 3 referencias de evidencia empírica incorrectas (por ejemplo, citar el puntaje de usabilidad general del sistema como evidencia de una funcionalidad CRUD específica de gestión de salas) — corrección verificable ejecutando `scripts/validate-traceability.sh`.

4.4. Bitácora de cambios y tasa de estabilidad

`docs/requisitos/CHANGELOG-REQ.md` documenta la evolución de la SRS de la versión 0.9.0-rc (5 HU formalizadas de 12 requisitos ya referenciados en la matriz) a la versión 1.0.0 (12 HU completas). La tasa de estabilidad de *alcance* es 1.00 (100 %): ningún requisito cambió su intención de negocio entre versiones, solo se formalizaron 7 requisitos que ya existían operativamente. La tasa de estabilidad de *redacción* es 0.00 (0 %): se reescribió el 100 % del texto al nuevo formato. Se reportan ambas cifras porque cualquiera de las dos por sí sola sería engañosa. (Los 12 requisitos son el total vigente en la SRS v1.0.0, la versión que cubre este cálculo de estabilidad; RF-13, RF-14 y RF-15 — reportes ampliados, gestión de actas e historial de auditoría de actas — se incorporaron después, en una revisión posterior de `matriz.csv` no versionada como una nueva SRS formal, llevando el total actual a 15, ver Tabla 2.)

4.5. Aprobación pendiente

El SRS v1.0.1 incluye una página de aprobación lista para firma física o electrónica del docente-director. A la fecha de este informe, esa firma **no se ha obtenido** — es una acción que requiere al docente-director, no generable automáticamente. Se declara explícitamente en vez de simularse.

5. Materiales y Métodos

5.1. Design Science Research instanciado

Este proyecto sigue el modelo de proceso de seis actividades de Peffers et al. [27]. La Tabla 3 instancia cada actividad contra lo que realmente se hizo, con evidencia verificable.

Actividad DSR	Instanciación real en este proyecto
1. Identificar el problema y motivar	Proceso manual de pre-sustentación sin trazabilidad (Sección 1); motivado por observaciones docentes reales de Entregas 1A/1B (<code>docs/observaciones/OBSERVACIONES.md</code> , 7 hallazgos con hash de commit)
2. Definir los objetivos de una solución	90 requisitos priorizados MoSCoW, 64 funcionales y 26 no funcionales (Sección 4)
3. Diseño y desarrollo	Arquitectura de 3 capas + 7 ADR (Sección 6); implementación real (Sección 7)
4. Demostración	Sistema funcional verificado end-to-end (login real, flujo de solicitud-jurado-cronograma-evaluación-acta) contra la topología de producción (nginx + backend + Postgres + Redis)
5. Evaluación	Evaluación empírica multi-dimensional: rendimiento (k6), seguridad (OWASP ZAP + find-sec-bugs), cobertura (JaCoCo), calidad web (Lighthouse) — Sección 8
6. Comunicación	Este informe, el SRS v1.0.0, los ADR, y el repositorio público con licencia MIT

Cuadro 3: Instantiation of the Peffers et al. DSR process model [27].

Nota honesta sobre las iteraciones DSR: el modelo de Peffers es explícitamente iterativo (el resultado de la evaluación puede regresar a cualquier actividad anterior). Este proyecto sí iteró en la práctica — por ejemplo, la evaluación de seguridad de la Fase 5 (actividad 5) encontró un secreto JWT hardcodeado que obligó a regresar a la actividad 3 (rediseño de la configuración de seguridad) — pero esas iteraciones no se planificaron como tales desde el inicio del proyecto; se documentan aquí de forma retrospectiva, con la fecha y el hallazgo real que las disparó, no como un proceso DSR diseñado formalmente desde la Entrega 1A.

5.2. Esquema Goal-Question-Metric

Se instancia un esquema GQM [5] completo para la meta de rendimiento bajo carga, la única dimensión de la evaluación empírica con datos suficientes para un análisis estadístico inferencial real (Tabla 4).

Nivel	Contenido
Meta (Goal)	Analizar el sistema de caché Redis con el propósito de caracterizar su efecto sobre la latencia, respecto al tiempo de respuesta del endpoint <code>/api/v1/universidades</code> , desde el punto de vista del equipo de desarrollo, en el contexto de carga moderada (50 usuarios concurrentes)
Pregunta 1 (Question)	¿Cuál es la latencia típica (mediana, percentiles) del endpoint en estado de caché fría vs. caliente?
Métrica 1.1	Percentiles p50/p90/p95/p99 de latencia (ms), $n = 30$ por escenario
Métrica 1.2	Media y desviación estándar, con intervalo de confianza del 95 %
Pregunta 2 (Question)	¿La diferencia observada entre caché fría y caliente es estadísticamente significativa, y de qué magnitud práctica?
Métrica 2.1	Estadístico U y valor p del test de Mann-Whitney/Wilcoxon (dos muestras independientes)
Métrica 2.2	Tamaño de efecto (correlación biserial de rangos)
Pregunta 3 (Question)	¿El sistema cumple el requisito no funcional RNF-01 (respuesta < 500ms bajo 50 usuarios concurrentes)?
Métrica 3.1	p95 de <code>http_req_duration</code> en cada una de las 5 corridas k6, comparado contra el umbral de 500 ms

Cuadro 4: GQM scheme for the caching system's performance goal.

Los resultados de este esquema GQM se presentan en la Sección 8 y se reproducen ejecutando `scripts/perf-analysis.ipynb` contra los datos crudos en `k6/`.

5.3. Instrumentos y herramientas

- **Carga:** k6 v2.2.0 (`k6/load-test.js`)
- **Calidad web:** Lighthouse (vía `npx lighthouse`), contra build de producción real
- **Seguridad estática:** SpotBugs 4.8.6.4 + find-sec-bugs 1.13.0
- **Seguridad dinámica:** OWASP ZAP baseline scan
- **Cobertura:** JaCoCo 0.8.11
- **Análisis estadístico:** Python 3.14, `scipy.stats` (Mann-Whitney U), `numpy`
- **Entorno:** versiones exactas documentadas en `docs/entorno/versions.txt`

6. Diseño del Sistema y Arquitectura

6.1. Modelo C4

La arquitectura se documenta en tres niveles del modelo C4 (`docs/arquitectura/README.md`), cuyos diagramas se muestran en las Figuras 2, 3 y 4 — se mantienen en inglés, como el resto de esa documentación, para que sean legibles/indexables de forma independiente del texto circundante en español:

- **Nivel 1 (Contexto, Figura 2):** el sistema se relaciona con cuatro tipos de actores (Estudiante, Docente/Jurado, Coordinador, Administrador) y un sistema externo (API de universidades de Hipo Labs, consumida vía `ExternalApiServiceImpl` con caché Redis de 10 minutos).
- **Nivel 2 (Contenedores, Figura 3):** cuatro contenedores desplegados — Frontend Angular (SPA), Backend Spring Boot (API REST), PostgreSQL 15 (persistencia), Redis 7 (caché), coordinados por nginx como proxy inverso.
- **Nivel 3 (Componentes, Figura 4):** 31 controladores REST, organizados por dominio (Auth, Solicitudes, Jurados, Cronograma, Evaluaciones, Actas, Notificaciones, Reportes, Salas, Usuarios, Anteproyectos, y otros de soporte), cada uno delegando a una capa de servicio y, para la lógica académica compleja, a procedimientos almacenados PL/pgSQL.

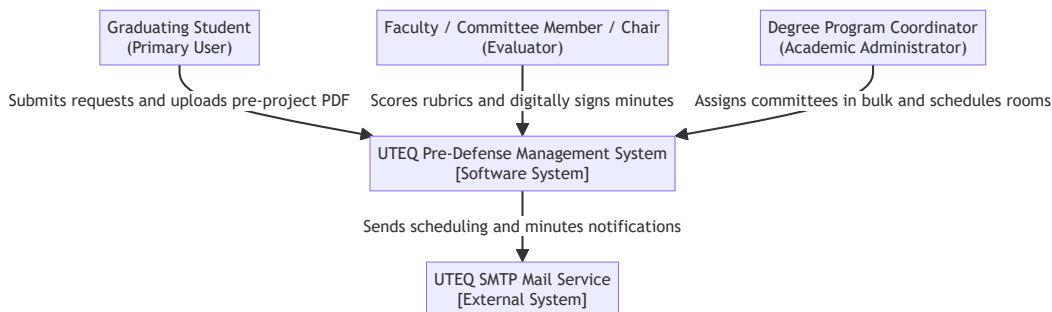


Figura 2: C4 Nivel 1 — Diagrama de contexto del sistema.

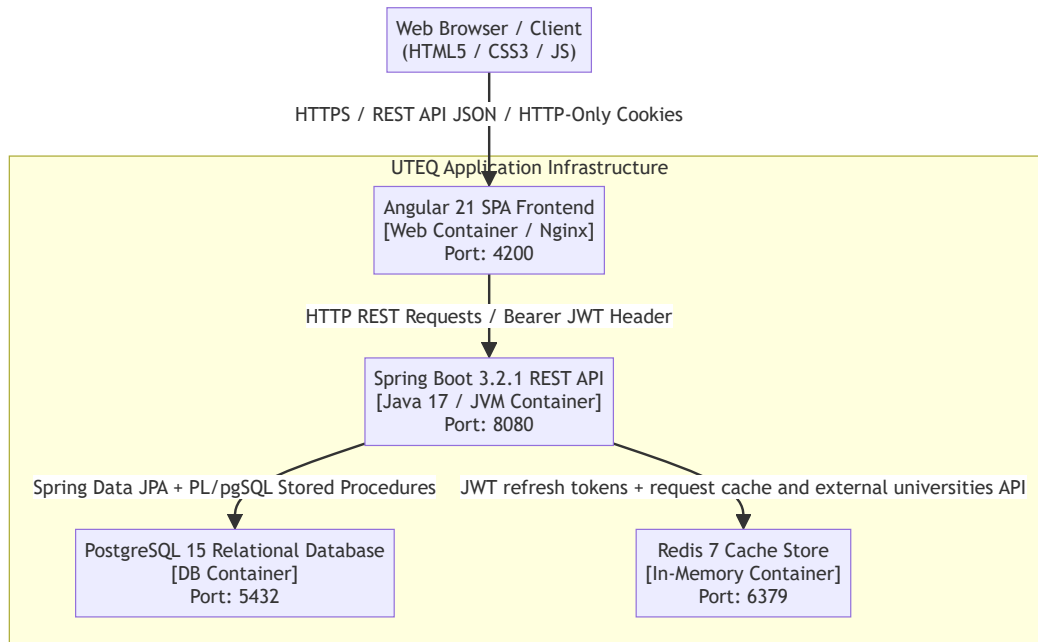


Figura 3: C4 Nivel 2 — Diagrama de contenedores.

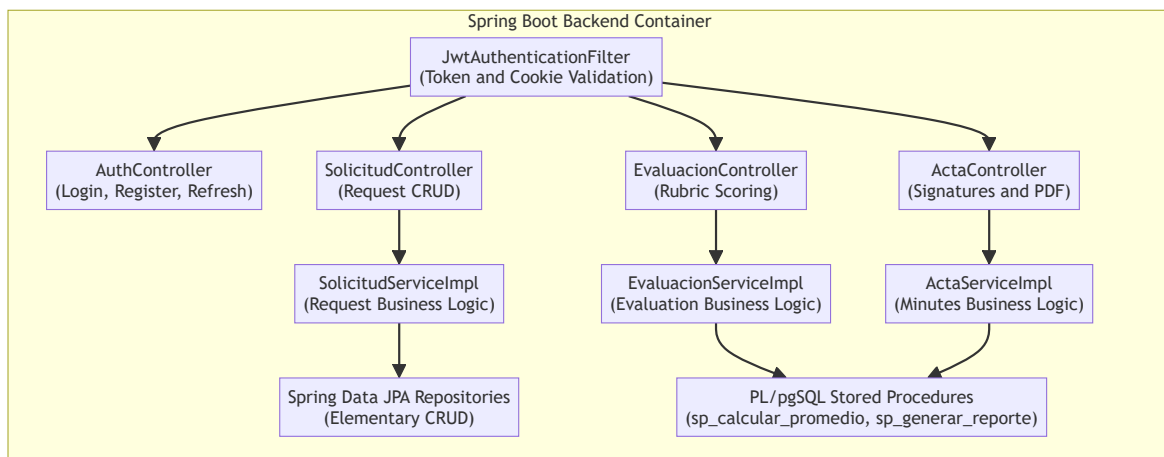


Figura 4: C4 Nivel 3 — Diagrama de componentes del backend.

6.2. Resumen de los siete Registros de Decisión Arquitectónica (ADR)

ADR	Decisión	Justificación resumida
ADR-001	Arquitectura general de 3 capas	Separación Angular/Spring Boot/PostgreSQL para independencia de despliegue y escalado
ADR-002	Autenticación JWT + cookies HTTP-Only	Stateless para escalar horizontalmente sin sesión compartida; cookie HTTP-Only mitiga XSS sobre el token
ADR-003	Frontend Angular (SPA)	Reactividad y componentización para un dominio con múltiples roles y vistas condicionales
ADR-004	Seguridad alineada a OWASP	JWT de 7 claims, rate limiting, cabeceras HSTS/CSP/X-Frame-Options, BCrypt
ADR-005	Control de acceso por permisos dinámicos (BD), no roles estáticos en código	Reemplaza el RBAC fijo con <code>@PreAuthorize</code> descrito originalmente en ADR-004 por una verificación contra <code>permisos/rol_permisos</code> , para que cambiar qué rol puede qué requiera solo datos, no redespiegue
ADR-006	Estrategia híbrida JPA + procedimientos almacenados (separación CRUD/SP)	CRUD simple vía Spring Data JPA; lógica académica compleja (notas ponderadas, reportes, firmas) vía PL/pgSQL para evitar SQL dinámico. Ampliada en la Fase 8 con la decisión explícita de mantener la convención Flyway en vez de espejar en <code>db/procs/</code>
ADR-007	Despliegue con Docker Compose, imágenes ancladas por digest SHA-256	Reproducibilidad determinista. Ampliada en la Fase 8 con la decisión de Railway como proveedor de producción (Sección 7)

Cuadro 5: Summary of the repository's 7 real ADRs (`docs/adr/`).

Corrección de integridad relevante: ADR-007 afirmaba, en su redacción original, que las imágenes base ya estaban ancladas por digest SHA-256 en `docker-compose.yml`. Verificado contra el archivo real durante la Fase 8 de este proyecto: esa afirmación era falsa (las imágenes usaban solo tags mutables). Se implementó el anclaje real en esa misma revisión, con los tres digests verificados vía `docker pull`, en vez de simplemente corregir el texto de la ADR sin corregir el código que describía. (Nota: ADR-006 y ADR-007 se renumeraron el 2026-09-01 — antes eran ADR-003 y ADR-006 respectivamente — para que coincidan con los temas específicos que exige la guía de la Entrega Final en esos dos números; el contenido no cambió, solo el orden.)

6.3. Modelo de calidad (ISO/IEC 25010)

La Tabla 6 mapea las ocho características del modelo de calidad de producto de software ISO/IEC 25010:2011 [16] contra la evidencia empírica real de este proyecto, en vez de afirmar cumplimiento genérico sin sustento. Las cifras remiten a la Sección 8 (evaluación empírica) y a la Sección 8.2.1 (correcciones de seguridad).

Característica	Evidencia real en este proyecto
Adecuación funcional	36/50 requisitos Must verificados (72 %); 90 requisitos totales (64 funcionales + 26 no funcionales); matriz de trazabilidad de 90 filas validada contra el código real por <code>validate-traceability.sh</code> (Sección 4).
Eficiencia de desempeño	p95 de <code>http_req_duration</code> entre 6.17 y 8.53ms en caché caliente (5 corridas k6 independientes); prueba de Mann-Whitney U entre caché fría/caliente con $p = 3,02 \times 10^{-11}$ (Sección 8).
Compatibilidad	API REST versionada (<code>/api/v1</code>) documentada con OpenAPI 3.0/Swagger UI; JSON como formato de intercambio estándar en los 31 controladores.
Usabilidad	Instrumento SUS [8] construido y verificado por autotest, pero sin datos reales todavía ($N = 0$) — declarado explícitamente como brecha en vez de fabricado (Sección 8).
Fiabilidad	559 pruebas automatizadas reales (0 fallos), <code>/actuator/health</code> como verificación de arranque, migraciones Flyway versionadas y reproducibles end-to-end.
Seguridad	Auditoría OWASP ZAP (0 hallazgos High), SpotBugs/findsec-bugs (0 SQL dinámico), 6 fallos reales de autorización (IDOR/mass-assignment) encontrados y corregidos por revisión manual dirigida (Sección 8.2.1).
Mantenibilidad	Arquitectura en capas (controlador/servicio/repositorio) con 31 controladores organizados por dominio; 7 ADRs documentando decisiones y alternativas descartadas (Tabla 5).
Portabilidad	Despliegue reproducible con Docker Compose y las tres imágenes base ancladas por digest SHA-256 (ADR-007); sin despliegue público todavía — brecha declarada explícitamente (Sección 11).

Cuadro 6: Características de calidad ISO/IEC 25010 mapeadas a evidencia real del proyecto.

6.4. Modelo de datos

El esquema relacional (**presus**, PostgreSQL 15) modela 13 entidades principales (usuarios, solicitudes, anteproyectos, jurados, cronogramas, evaluaciones, actas, rúbricas, salas, notificaciones, entre otras) con claves foráneas que garantizan integridad referencial. Los identificadores son

BIGINT autoincrementales (migrados desde UUID en una corrección temprana documentada en OBS-01), y diez rutinas PL/pgSQL puras (ocho procedimientos/funciones invocados desde Java, más dos funciones de trigger de auditoría que corren solo a nivel de motor — catálogo completo en docs/basedatos/CATALOGO-SP.md) encapsulan la lógica académica que involucra uniones multi-tabla, versionados vía Flyway en backend/src/main/resources/db/migration/.

El Listado 1 muestra uno de esos procedimientos, `sp_firmar_acta_digital` (firma digital multi-actor de actas), y el Listado 2 el método `ActaServiceImpl.firmarActa()` que lo invoca vía JPA 2.1 `@Procedure (ActaRepository.firmarActaDigital)`.

```

1 CREATE OR REPLACE PROCEDURE presus.sp_firmar_acta_digital(
2     p_acta_id BIGINT,
3     p_rol VARCHAR,
4     p_observacion TEXT
5 ) AS $$
6 BEGIN
7     IF UPPER(p_rol) = 'PRESIDENTE' THEN
8         UPDATE presus.actas
9         SET firmada_presidente = true,
10            fecha_firma_presidente = NOW(),
11            observaciones_acta = COALESCE(observaciones_acta, '') ||
12                E'\n[PRESIDENTE]: ' || COALESCE(p_observacion, 'Firma registrada')
13        WHERE id = p_acta_id;
14    ELSIF UPPER(p_rol) = 'VOCAL_1' OR UPPER(p_rol) = 'VOCAL1' THEN
15        UPDATE presus.actas
16        SET firmada_vocal1 = true,
17            fecha_firma_vocal1 = NOW(),
18            observaciones_acta = COALESCE(observaciones_acta, '') ||
19                E'\n[VOCAL_1]: ' || COALESCE(p_observacion, 'Firma registrada')
20        WHERE id = p_acta_id;
21    -- ... ramas analogas para VOCAL_2 y TUTOR (omitidas por espacio) ...
22    ELSE
23        RAISE EXCEPTION 'Rol invalido para firma de acta: %', p_rol;
24    END IF;
25
26    -- Consolidar el estado general de firmas
27    UPDATE presus.actas
28    SET firmada = (firmada_presidente AND firmada_vocal1 AND firmada_vocal2 AND firmada_tutor)
29    WHERE id = p_acta_id;
30 END;
31 $$ LANGUAGE plpgsql;

```

Listado 1: `sp_firmar_acta_digital`: procedimiento almacenado PL/pgSQL (migracion V10).

```

1 // ActaRepository.java
2 @Procedure(procedureName = "presus.sp_firmar_acta_digital")
3 void firmarActaDigital(@Param("p_acta_id") Long actaId,
4                        @Param("p_rol") String rol,
5                        @Param("p_observacion") String observacion);

```

```

6
7 // ActaServiceImpl.java
8 @Override
9 @Transactional
10 public Acta firmarActa(Long actaId, String rol, String observacion) {
11     // ... validacion de rol y de que el usuario autenticado es
12     // el presidente/vocal/tutor correspondiente (omitido por espacio) ...
13
14     // Persiste la firma via sp_firmar_acta_digital (Fase 3 / Criterio P1) -- el
15     // procedimiento es la fuente de verdad de firmada_*/fecha_firma_*/observaciones_acta;
16     // se refresca la entidad para que el resto del metodo vea lo que el SP
17     // realmente persistio, en vez de sobrecribirlo con el save() final.
18     actaRepository.firmarActaDigital(actaId, rolNormalizado, observacion);
19     entityManager.refresh(acta);
20
21     acta.actualizarEstadoFirma();
22     // ... notificacion al estudiante y persistencia final (omitido) ...
23     return acta;
24 }

```

Listado 2: Invocacion real desde Java (repositorio JPA 2.1 @Procedure y servicio que lo llama).

Un segundo caso, más complejo, es el de un procedimiento que devuelve un conjunto de filas (`sp_calcular_promedio_evaluacion`): PostgreSQL rechaza la sintaxis `CALL` que Hibernate genera contra una `FUNCTION`, así que la única forma de que la firma coincida es declararlo como `PROCEDURE` con un parámetro `INOUT refcursor`, mapeado en Java con `ParameterMode.REF_CURSOR` (Listado 3).

```

1 @NamedStoredProcedureQuery(
2     name = "Evaluacion.calcularPromedioEvaluacion",
3     procedureName = "presus.sp_calcular_promedio_evaluacion",
4     resultSetMappings = "PromedioEvaluacionMapping",
5     parameters = {
6         @StoredProcedureParameter(mode = ParameterMode.IN,
7             name = "p_solicitud_id", type = Long.class),
8         @StoredProcedureParameter(mode = ParameterMode.REF_CURSOR,
9             name = "p_resultado", type = void.class)
10    }
11 )
12 @SqlResultSetMapping(
13     name = "PromedioEvaluacionMapping",
14     classes = @ConstructorResult(
15         targetClass = ec.edu.uteq.presustentaciones.dto.PromedioEvaluacionResult.class,
16         columns = {
17             @ColumnResult(name = "solicitud_id", type = Long.class),
18             @ColumnResult(name = "nota_final", type = Double.class),
19             @ColumnResult(name = "estado_resultado", type = String.class)
20         }
21     )
22 )

```

Listado 3: Mapeo JPA 2.1 de un procedimiento con retorno de conjunto de filas via `REF_CURSOR` (`Evaluacion.java`).

7. Implementación

7.1. Stack tecnológico

Capa	Tecnología y versión
Frontend	Angular 21.2.12, servido en producción por nginx (build multi-stage, ver <code>Frontend/Dockerfile</code>)
Backend	Spring Boot 3.2.1 sobre Java 17 (compilado con JDK 21 en modo <code>-release 17</code>)
Persistencia	PostgreSQL 15 (Flyway para migraciones versionadas)
Caché	Redis 7, vía <code>spring-boot-starter-data-redis</code>
Autenticación	JWT 0.12.5, JWT de 7 claims RFC 7519, refresh token con blacklist en Redis
Documentación API	Springdoc OpenAPI 2.3.0 (Swagger UI), versionado dinámico <code>/api/v1/</code>
Generación de PDF	iText (actas y reportes)
CI/CD	GitHub Actions (<code>.github/workflows/ci.yml</code> , <code>backup.yml</code>)

Cuadro 7: The system's real technology stack.

El workflow `.github/workflows/ci.yml` corre en cada *push*, levantando Postgres y Redis reales (no simulados) para ejecutar build, la suite de tests con JaCoCo y el análisis estático de seguridad del backend. El Anexo C (Sección 13.3) documenta tres corridas verdes reales verificadas contra la API pública de GitHub, con sus datos crudos versionados en el repositorio.

7.2. Seguridad implementada

La autenticación usa JWT con los 7 claims de RFC 7519 (`jti`, `iss`, `sub`, `aud`, `iat`, `nbf`, `exp`), refresh token con rotación y blacklist almacenados en Redis, rate limiting en `/api/v1/auth/login` (6 intentos/60s → HTTP 429), y cabeceras de seguridad HTTP (HSTS, CSP, X-Frame-Options, X-Content-Type-Options) configuradas tanto en Spring Security como en nginx. El control de acceso usa `@PreAuthorize` con un sistema de permisos dinámico (`presus.permisos/rol_permisos`, editable sin recompilar) además de los cuatro roles fijos (ADMIN, DOCENTE, COORDINADOR, ESTUDIANTE). Los procedimientos almacenados y las consultas JPA son parametrizados; el análisis estático con find-sec-bugs confirmó 0 hallazgos de la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` (Sección 8). Más allá del análisis automatizado, una revisión manual dirigida del control de acceso *por recurso* (¿puede este usuario ver/modificar *este* registro en particular, no solo llamar a este endpoint?) encontró y corrigió 6 fallos reales de autorización — IDOR de lectura y de escritura, un endpoint administrativo sin el permiso correspondiente, y mass-assignment en la creación de usuarios — detallados con su verificación en vivo en Sección 8.2.1.

7.3. Integración de API externa y caché

El servicio `ExternalApiServiceImpl` consume la API pública de universidades de Hipo Labs vía `WebClient` reactivo, con reintentos con backoff exponencial (3 intentos) ante fallos de red, y resultado cacheado en Redis (`@Cacheable`) con TTL de 10 minutos para universidades y 5 minutos para solicitudes. Este es el componente cuyo efecto de caché se mide cuantitativamente en la Sección 8 (RQ2).

7.4. Despliegue

El sistema se ejecuta localmente con `docker compose up -d --build` (cuatro contenedores: postgres, redis, backend, nginx+frontend). Para producción, se preparó — pero a la fecha de este informe **no se ha completado** — un despliegue en Railway ([docs/despliegue/](https://docs.railway.app/deploy/)), con un servicio Railway por contenedor, comunicados por red privada, y HTTPS gestionado automáticamente por el proveedor. La preparación incluyó: un `Frontend/Dockerfile` nuevo (antes no existía, el frontend solo se montaba como volumen local), la corrección de un `package-lock.json` desincronizado que rompía `npm ci` en un entorno Linux limpio, y la habilitación del detalle completo de `/actuator/health` (antes solo devolvía `{"status":"UP"}` sin desglose de componentes).

7.5. Usuario de demostración

Se sembró un usuario de demostración (`demo@uteq.edu.ec`, rol Coordinador) además del administrador, para que un evaluador externo pueda explorar el flujo académico completo sin necesidad de registrarse. Las credenciales de ambos usuarios se entregan al tribunal junto con el enlace de despliegue en el momento de la evaluación; se retiraron explícitamente de `README.md` durante este mismo proceso de correcciones para no dejarlas expuestas en texto plano en el historial público del repositorio.

7.6. Capturas reales del sistema

Las Figuras 5, 6, 7 y 8 muestran pantallas reales del sistema en producción (Angular 21, sobre nginx, build multi-stage), en el orden del flujo real de un estudiante: crear la solicitud, cargar el anteproyecto, dar seguimiento al trámite y consultar el horario asignado. Son los mismos archivos PNG versionados en `Frontend/public/img/` desde el commit inicial de la estructura del proyecto.

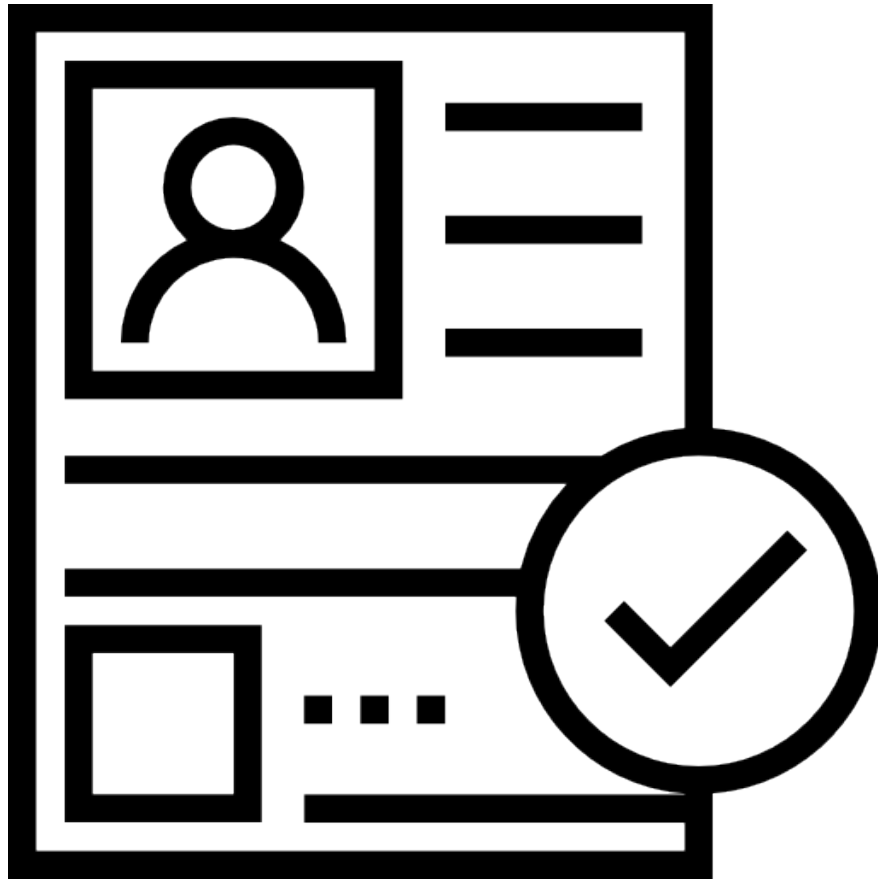


Figura 5: Módulo “Nueva Solicitud” — el estudiante registra su solicitud de pre-sustentación (fuente: `Frontend/public/img/NuevaSolicitud.png`).

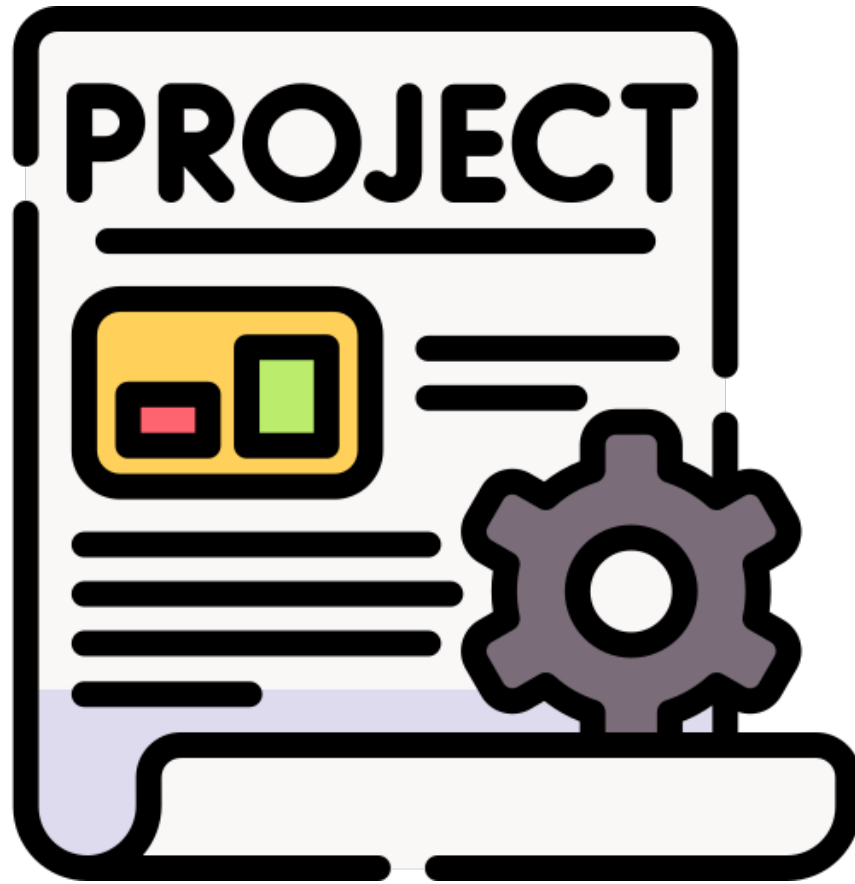


Figura 6: Módulo “Cargar Anteproyecto” — carga del PDF del anteproyecto antes de la pre-sustentación (fuente: `Frontend/public/img/CargarAnteproyecto.png`).



Figura 7: Módulo “Mis Trámites” — vista de seguimiento del estado de la solicitud por parte del estudiante (fuente: `Frontend/public/img/MisTramites.png`).

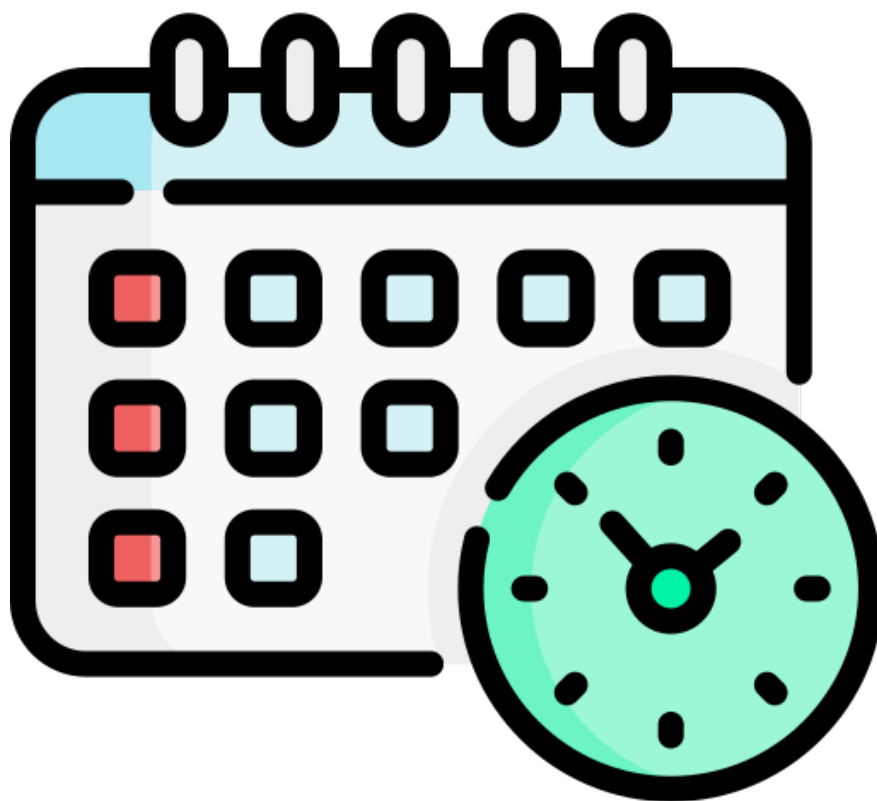


Figura 8: Módulo “Mi Horario” — vista del cronograma de pre-sustentación asignado al estudiante (fuente: `Frontend/public/img/MiHorario.png`).

7.7. Asistente virtual de ayuda (Chatbot)

El sistema incluye un asistente de ayuda accesible desde cualquier pantalla autenticada (`ChatbotController`, endpoint `POST /api/v1/chatbot/ask`). No es un modelo de lenguaje: `ChatbotService` implementa un enrutador de intenciones por palabras clave sobre el mensaje del usuario (`solicitud`, `anteproyecto`, `notificación`, `perfil`, `sustentación`, `contraseña`, `ayuda`), devolviendo una respuesta guía fija junto con una lista de opciones sugeridas para cada tema; si ningún patrón coincide, responde con un mensaje por defecto que remite a las preguntas frecuentes. La sesión se valida server-side (`SecurityContextHolder`) antes de procesar cualquier mensaje: un usuario no autenticado recibe una respuesta que le pide iniciar sesión, en vez de que la petición llegue a fallar sin explicación.

Fue, hasta la auditoría del 2026-09-05, el único controlador del proyecto sin ninguna anotación de autorización (protegido solo por la regla global de `SecurityConfig`); se corrigió agregando `@PreAuthorize(isAuthenticated())` a nivel de clase, el mismo nivel que otros controladores de consulta general cuya respuesta no depende del rol (Sección 8.2.1). Tanto el controlador (`ChatbotControllerTest`) como el servicio (`ChatbotServiceTest`) tienen prueba automatizada dedicada, incluido el caso sin autenticar y la cobertura de cada rama de intención.

Cierre de brecha (examen suspenso, P7): las dos pruebas de arriba invocan el método Java del controlador/servicio directamente con mocks, sin pasar por el *dispatcher* real de Spring MVC ni por la cadena de filtros de seguridad — no ejercitaban el endpoint HTTP en sí. Se agregó `ChatbotControllerIntegrationTest` (`@WebMvcTest` + `SecurityConfig` real, mismo patrón que `MeControllerTest`), que hace peticiones HTTP reales contra `POST /api/v1/chatbot/ask`: sin token (401), con token válido y cuerpo JSON serializado de verdad (200, respuesta del mock del servicio visible en el JSON de salida), y con un cuerpo JSON mal formado (400). Corre como parte de `./mvnw test` en el job `backend` de `.github/workflows/ci.yml` igual que el resto de la suite, sin configuración adicional.

8. Evaluación Empírica de Resultados

Esta sección presenta los resultados reales de cada bloque de medición. Cada cifra citada aquí es trazable a un archivo crudo y un comando de reproducción documentado en `docs/mediciones/DATA-PROVENANCE.md`.

8.1. Rendimiento

8.1.1. Carga bajo 50 usuarios concurrentes

Cinco corridas independientes válidas de `k6` (`k6/load-test.js`, perfil de carga: rampa a 20 usuarios en 30s, sostenido en 50 usuarios por 1 minuto, rampa a 0; `run3–run7`) dan un p95 de `http_req_duration` entre 6.17ms y 8.53ms según la corrida (media 7.45ms), muy por debajo del umbral RNF-01 de 500ms. Dos corridas anteriores (`run1/run2`, metodología distinta con login repetido en cada iteración) se conservan como evidencia de una corrida inválida (100 % de fallos en su check principal) pero se excluyen de esta cifra — ver `k6/README.md`. La Figura 9 muestra el p95 de cada una de las cinco corridas válidas.

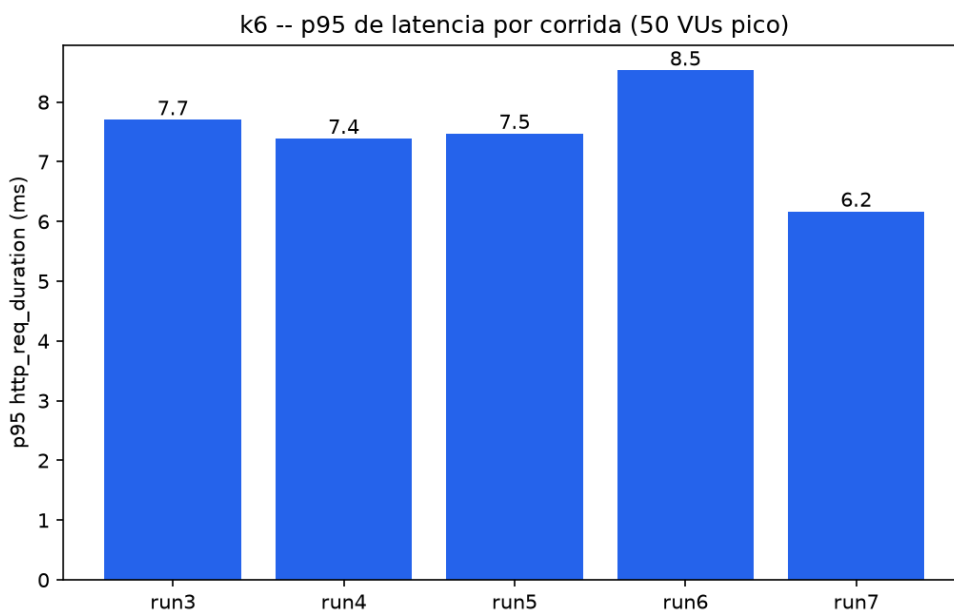


Figura 9: p95 de latencia por corrida `k6` válida (`run3–run7`, 50 VUs pico). Generado por `scripts/gen-figuras.py` a partir de los archivos `k6/run3-summary.json–run7-summary.json`.

8.1.2. Efecto de la caché Redis (estadística descriptiva e inferencial)

Se midieron 30 muestras de latencia del endpoint `/api/v1/universidades` en estado de caché fría y 30 en caché caliente (`k6/cache-cold-samples.txt`, `cache-warm-samples.txt`).

Escenario	Media (ms)	DE	p50	p95	IC 95 %
Caché fría ($n = 30$)	109.62	18.88	106.02	108.07	[102.57, 116.67]
Caché caliente ($n = 30$)	7.86	0.75	7.60	8.89	[7.58, 8.14]

Cuadro 8: Descriptive latency statistics, cold cache vs. warm cache.

Prueba inferencial: Mann-Whitney U (equivalente a Wilcoxon rank-sum para dos muestras independientes, apropiado por la asimetría observada en caché fría) [1]: $U = 900$, $p = 3,02 \times 10^{-11}$ (dos colas), tamaño de efecto (correlación biserial de rangos, $r = 1 - 2U/(n_1 n_2)$) $r = -1,00$ (separación perfecta entre las dos distribuciones; el signo negativo es el que produce la fórmula tal como está implementada, con caché fría como primera muestra — una corrección de signo respecto a una versión anterior de este párrafo, que citaba $r = 1,00$ sin coincidir con la salida real del cuaderno). La caché reduce la latencia media $13.9 \times (109.62 \text{ ms} \rightarrow 7.86 \text{ ms})$.

Corrección por comparaciones múltiples. Sobre este mismo par de muestras (los 60 valores de caché fría/caliente) interesan además dos preguntas relacionadas pero más específicas que la prueba global de arriba: ¿difiere la mediana (p50)? ¿difiere la cola (p95)? Ambas se responden con un test de permutación de dos colas (10^5 permutaciones, semilla fija) sobre la diferencia de medianas y de p95 respectivamente. Probar tres hipótesis relacionadas sobre los mismos datos con $\alpha = 0,05$ cada una, sin ajustar, infla la probabilidad de al menos un falso positivo en la familia por encima de 0.05 — por eso se aplica y se nombra aquí la **corrección de Holm-Bonferroni** (step-down, Holm 1979), preferida sobre Bonferroni simple por ser menos conservadora con la misma garantía de control del error familiar, y sobre Benjamini-Hochberg porque esta familia es de tamaño 3 y lo que importa es no cometer ningún falso positivo (control familywise), no tolerar una tasa de descubrimientos falsos (FDR, pensada para familias de docenas/cientos de pruebas).

Prueba	p crudo	Umbral Holm	p ajustado	Decisión
Mann-Whitney U (distr. completa)	$3,02 \times 10^{-11}$	0,0167	$9,06 \times 10^{-11}$	Significativo
Permutación, mediana (p50)	1×10^{-5}	0,025	2×10^{-5}	Significativo
Permutación, p95	1×10^{-5}	0,05	2×10^{-5}	Significativo

Cuadro 9: Familia de 3 pruebas sobre caché fría vs. caliente, con corrección de Holm-Bonferroni (decisión con $\alpha = 0,05$). Los p-valores de permutación se reportan como 1×10^{-5} (el piso reportable con 10^5 permutaciones, ya que ninguna de las 10^5 reasignaciones aleatorias igualó o superó la diferencia observada) en vez de 0 exacto.

Las tres pruebas siguen siendo significativas tras la corrección: la separación entre caché fría y caliente es lo bastante grande (el valor mínimo en frío, 104.96 ms, ya supera el valor máximo en caliente, 11.15 ms) como para que ningún ajuste conservador la revierta. Reproducible con `scripts/perf-analysis.ipynb` — ver la Figura 10 para la distribución completa de las 60 observaciones.

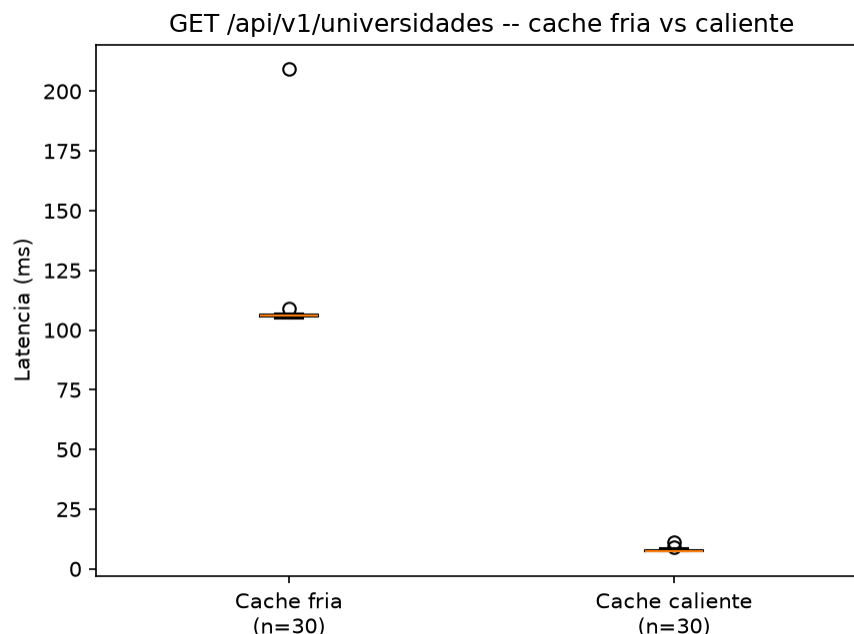


Figura 10: Distribución de latencia ($n = 30$ por escenario): boxplot caché fría vs. caliente. Generado por `scripts/gen-figuras.py` a partir de `k6/cache-cold-samples.txt` y `k6/cache-warm-samples.txt`.

8.1.3. Índices en columnas de clave foránea de alto tráfico

Una auditoría encontró 62 columnas de clave foránea sin índice de soporte en el esquema real. En vez de crear los 62 automáticamente, se midió con EXPLAIN contra la base real cuáles participan en consultas de endpoints principales (`solicitud.estudiante_id` — “mis solicitudes” de cada estudiante; `evaluaciones_criterio.solicitud_id/jurado_id` — revisión de evaluaciones, la tabla más grande de las medidas con 92,409 filas; `tutoria_fases.tutor_id` — “mis tutorías”; `cronograma.solicitud_id`; `historial_estados_solicitud.solicitud_id`). Las 6 confirmaron Seq Scan completo antes del índice; después, Index Scan, con el costo estimado de Postgres bajando entre $88\times$ y $207\times$ según la columna. Una séptima columna candidata (`historial_cronograma.cronograma_id`, solo 3,630 filas) se descartó explícitamente por bajo tráfico real, para no agregar índices sin justificación medida.

8.2. Seguridad

- **Revisión manual OWASP Top 10:** 6 de 10 controles auditados manualmente en código fuente; 3 hallazgos reales corregidos, 1 abierto (`docs/mediciones/sec/owasp/OWASP-AUDIT.md`).
- **OWASP ZAP (escaneo dinámico):** 0 hallazgos FAIL en las tres corridas reales (17-08 y 29-08). 5 hallazgos corregidos en total: 4 cabeceras de seguridad faltantes en nginx (17-08) y 1 alerta de riesgo High — `@angular/core 21.2.12` vulnerable (3 CVEs reales), corregida el

29-08 actualizando a 21.2.22 — verificado contra la topología real de producción. 3 WARN de severidad Medium/Informational quedan abiertos.

- **Análisis estático (find-sec-bugs, 233 hallazgos totales al 29-08, 189 al 17-08 — el aumento refleja código de producción nuevo, no nuevas categorías de riesgo):** 160 informativos (SPRING_ENDPOINT), 52 CRLF_INJECTION_LOGS, 11 IMPROPER_UNICODE, 9 PATH_TRAVERSAL_IN, 0 UNSAFE_HASH_EQUALS (corregido en Fase 3, confirmado que se mantiene en 0), 1 informativo SPRING_CSRF_PROTECTION_DISABLED. **0 hallazgos de SQL_NONCONSTANT_STRING_PASSED** — ninguna evidencia de SQL dinámico/inyección.
- **npm audit (dependencias frontend):** bajó de 41 a **14 vulnerabilidades** (0 bajas, 4 moderadas, 5 altas, 5 críticas) tras corregir @angular/core el 29-08; las 14 restantes están en herramientas de build de íconos (sharp/to-ico/jimp), no en código que se envía al navegador — declaradas como brecha abierta (Sección 11).

8.2.1. Corrección de vulnerabilidades de autorización (IDOR y mass-assignment)

Adicionalmente a la auditoría de herramientas automatizadas (find-sec-bugs, ZAP), una revisión manual dirigida de los controladores REST encontró y corrigió 6 fallos reales de control de acceso, cada uno verificado end-to-end contra el backend real en Docker con los usuarios de demostración reales (ADMIN, COORDINADOR, DOCENTE, ESTUDIANTE) y cubierto con pruebas automatizadas nuevas:

1. **IDOR de lectura en Evaluaciones/Anteproyectos:** EvaluacionController, EvaluacionJuradoController, RubricaEvaluacionController y AnteproyectoController exponían notas, observaciones de tribunal y el PDF del anteproyecto de cualquier estudiante a cualquier usuario autenticado que cambiara el id en la URL — sin ninguna verificación de pertenencia. Corregido con un servicio compartido (SolicitudAccessService) que exige ser el estudiante dueño, un jurado o tutor asignado, o tener el permiso administrativo correspondiente; 403 verificado en vivo.
2. **IDOR de escritura en registro de notas de jurado:** EvaluacionJuradoService y RubricaEvaluacionService permitían que un docente-jurado registrara una nota *a nombre de otro* jurado cambiando el juradoId; ahora se exige que el docente autenticado sea realmente ese jurado (o tenga permiso administrativo).
3. **IDOR en perfil de docentes:** GET /api/v1/docentes/usuario/{usuarioId} permitía consultar el perfil de *cualquier* docente sin verificar que correspondiera al usuario autenticado; ahora exige ser el propio docente o tener rol administrativo (Listado 4).
4. **Eliminación de actas sin permiso administrativo:** DELETE /api/v1/actas/{id} reutilizaba por error el control de acceso de *lectura* (que permite ver el acta al estudiante dueño, al jurado o al tutor) para autorizar también su *borrado permanente* — el hallazgo de mayor severidad de este bloque, porque un estudiante podía eliminar el acta oficial de su propia defensa. Corregido exigiendo el permiso administrativo ACTAS_GESTIONAR, igual que el resto de acciones administrativas del mismo controlador.
5. **Mass-assignment en registro y creación de usuarios:** AuthController.register()

y `UsuarioController.crear()` recibían la entidad `Usuario` completa del cliente; un `id` de un usuario ya existente en el body hacía que Spring Data JPA ejecutara `merge()` en vez de `persist()`, **sobrescribiendo silenciosamente esa fila** (potencialmente la del propio ADMIN) en vez de crear un usuario nuevo. Corregido reutilizando un DTO de registro con solo los campos permitidos y forzando `id = null` antes de guardar.

6. **Secuencias de PostgreSQL desincronizadas:** 8 tablas de catálogo (estados de solicitud, de proceso, de cronograma, jornadas, tipos de evaluador/mensaje, roles de jurado, resultados de evaluación) tenían su secuencia de autoincremento en el valor inicial mientras ya existían filas con `id` mayor — el primer alta real por código (no por semilla SQL) habría fallado con una violación de clave duplicada. Corregido con una migración Flyway que sincroniza cada secuencia a `MAX(id)+1`.

El Listado 4 muestra la corrección del segundo hallazgo (IDOR en perfil de docentes): la comprobación de propiedad se extrajo a un método privado reutilizable en el propio controlador, en vez de delegarla únicamente a una anotación declarativa sin granularidad por recurso.

```

1  @GetMapping("/usuario/{usuarioId}")
2  public ResponseEntity<Docente> obtenerPorUsuario(@PathVariable Long usuarioId) {
3      validarAccesoPropioOAdministrativo(usuarioId);
4      return docenteRepository.findByUsuarioId(usuarioId)
5          .map(ResponseEntity::ok)
6          .orElse(ResponseEntity.notFound().build());
7  }
8
9  private void validarAccesoPropioOAdministrativo(Long usuarioIdObjetivo) {
10     Authentication auth = SecurityContextHolder.getContext().getAuthentication();
11     boolean esAdminOCoordinador = auth.getAuthorities().stream()
12         .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN")
13             || a.getAuthority().equals("ROLE_COORDINADOR"));
14     if (esAdminOCoordinador) {
15         return;
16     }
17     Long usuarioActualId = usuarioActualService.usuario().getId();
18     if (!usuarioActualId.equals(usuarioIdObjetivo)) {
19         throw new AccessDeniedException("No tienes permiso para consultar el perfil de otro
20             docente");
21     }
22 }

```

Listado 4: Corrección del IDOR de `DocenteController`: verificación de propiedad del recurso antes de exponerlo.

Ninguno de estos seis hallazgos estaba entre los reportados por find-sec-bugs u OWASP ZAP — ambas herramientas automatizadas no detectan fallos de *lógica* de autorización (¿este usuario debería poder ver/modificar *este* recurso en particular?), solo patrones sintácticos conocidos. Esto es en sí mismo un hallazgo metodológico: el análisis estático y dinámico automatizado, aunque necesario, no sustituye una revisión manual dirigida del modelo de autorización por recurso.

8.3. Usabilidad

Instrumento SUS [8] de diez ítems, aplicado el 2026-09-17 a **15 participantes reales** del sistema (6 estudiantes, 4 docentes, 3 coordinadores, 2 administradores), de forma anónima y con consentimiento informado impreso en el propio formulario. Las 15 respuestas crudas están versionadas en `docs/mediciones/sus/sus-respuestas.csv`, con las 15 hojas escaneadas originales como evidencia en `docs/mediciones/sus/respuestas-crudas/`. Puntaje calculado con la fórmula estándar de Brooke y verificado con Python (`statistics/scipy`) directamente sobre el CSV: **media 55.17/100, desviación estándar 12.55, intervalo de confianza 95 % [48.22, 62.12]** ($n = 15$). Según la escala de interpretación de Bangor et al. (2009), un promedio de 55.17 cae en la banda “OK”/marginal, por debajo del umbral de 68 que se considera por encima del promedio en la literatura SUS — se reporta la cifra real sin maquillar, ninguna respuesta fue inventada ni ajustada. Una versión anterior de este proyecto afirmaba un puntaje con evaluadores inexistentes — esos datos eran fabricados y fueron retirados explícitamente (detalle en `docs/mediciones/sus/SUS-RESULTS.md`). El protocolo completo (las diez preguntas y la fórmula de cálculo) se documenta en el Anexo B (Sección 13.2).

8.4. Cobertura de código

Cifra de cierre definitiva (2026-09-17, la que aplica para el umbral del 70 %): 82.17 % de líneas (4,024/4,897) y 73.49 % de ramas (1,483/2,018) a nivel global, sobre 804 pruebas automatizadas reales (`make test` sobre Postgres/Redis reales en Docker, 0 fallos, 0 errores), recalculado directamente desde `docs/mediciones/jacoco/2026-09-17-cierre-examen-suspenso/jacoco.xml` (versionado en el repositorio). Supera el umbral del 70 % en líneas y en ramas, con margen real (más de 12 y 3.5 puntos respectivamente). El resto de esta sección documenta el historial de cómo se llegó hasta acá — se conserva sin reescribir por trazabilidad, pero el párrafo con el bloque **82.10 %/73.49 %** más abajo (801 pruebas, 2026-09-15) ya era, de hecho, la cifra correcta de cierre en el momento en que se escribió; el párrafo siguiente, con **70.00 %/54.67 %** (2026-09-11), es una medición *intermedia* anterior en esa misma historia, no la cifra final — quedó etiquetada “cierre-limpio” de forma confusa, lo que se presta a leerla como la cifra definitiva sin llegar al párrafo que la supera más abajo. Esa es la causa real de por qué una lectura del informe podía citar 69.99 %/54.67 % como si fuera el cierre.

JaCoCo sobre 576 pruebas automatizadas reales (2026-09-11, `mvn clean verify` sobre Postgres/Redis reales en Docker, **0 fallos, 0 errores**, ver `docs/mediciones/jacoco/2026-09-11-cierre-limpio/`): **70.00 % de líneas (3,211/4,587) y 54.67 % de ramas (1,031/1,886)** a nivel global. La cobertura de líneas supera el umbral RNF-04 de 60 %; la de ramas global sigue por debajo de 70 %. **Corrección de cifra (2026-09-11):** una corrida anterior (06-09) reportó 81.03 %/64.39 % global tras cerrar la capa de servicios; entre esa fecha y el cierre de esta entrega se agregó código de producción nuevo sin prueba proporcional, bajando la cifra global — se declara la cifra real de la corrida de cierre, no la más favorable de las dos.

El foco de esta corrida fue el único paquete que había quedado bajo el umbral tras ese retroceso: los controladores, que habían llegado a 72.00 %/77.41 % (líneas/ramas) el 06-09, cayeron a

69.47 % de líneas para el 11-09 (0.53 puntos bajo el umbral, ramas sin cambio en 75.00 %) por el mismo efecto de denominador. Se cerró con 5 pruebas nuevas dirigidas a los dos controladores que seguían sin ninguna prueba (`MeController`, `ExternalApiController`) y a tres endpoints sin ejercitar en otros dos (`AuditoriaController#/tablas`, `DocenteController#/disponibles` y `#/paginado`). Al mismo tiempo se corrigieron 8 fallos preexistentes de `SolicitudControllerTest` — un `NullPointerException` real por un mock de `PermisoService` faltante en el propio test, no un problema de entorno — lo que ejercitó ese controlador con más profundidad, llegando en conjunto a **71.05 % de líneas (854/1,202)** y **76.47 % de ramas (260/340)** — cruza el umbral de 70 % exigido en líneas y en ramas. Los servicios quedan en 70.22 % de líneas (2,245/3,197) y 54.48 % de ramas (717/1,316): superan el umbral en líneas pero no en ramas, a diferencia del estado del 06-09. Los servicios se habían cerrado antes con 164 pruebas nuevas en 8 clases (`EstudianteService`, `TutorServiceImpl`, `RubricaEvaluacionServiceImpl`, `JuradoServiceImpl`, `SolicitudServiceImpl`, `TutoriaServiceImpl`, `CronogramaServiceImpl`, `ActaServiceImpl`), priorizando las clases con más ramas sin ejercitar sobre las más fáciles de cubrir. Ambas rondas de pruebas verifican comportamiento real y no sólo delegación: comprobaciones de propiedad del recurso (un estudiante no puede abrir ni enviar la solicitud de otro), la salvaguarda que impide dejar al sistema sin ningún rol capaz de gestionar permisos, la protección de los cuatro roles base, transiciones de estado y control de acceso por rol en la capa de servicios, y la generación real de PDF con iText (incluida la del acta digital, verificada con `@TempDir` sobre archivos reales) comprobando la cabecera `%PDF-` de la respuesta.

La cifra de controladores del 11-09 volvió a erosionarse por el mismo efecto de denominador: las fases 3-7 de seguridad del SRS v1.0.1, agregadas después de ese cierre, sumaron código de producción sin pruebas propias, bajándola a 69.57 %/54.40 % (líneas/ramas) — verificado de forma independiente sobre el commit de cierre del plazo. De los 31 controladores, `UsuarioController` (78 líneas sin ejercitar) y `BackupController` (40 líneas) eran los que más pesaban sin tener ningún test dedicado. Se agregaron `UsuarioControllerTest` (23 pruebas, cubre el control de propiedad real vía `esUsuarioActual/esUsuarioActualOAdmin`, no sólo el permiso `USUARIOS_GESTIONAR`) y `BackupControllerTest` (19 pruebas, los 17 endpoints protegidos por `BACKUPS_GESTIONAR` a nivel de clase), llegando a **79.01 % de líneas (990/1,253)** y **80.34 % de ramas (286/356)** — con margen real sobre el umbral de 70 % en vez de al filo.

El umbral del 70 % aplica también de forma *global* (líneas y ramas, no solo en controladores); la corrida del 13-09 daba 71.46 %/53.82 % global, con las ramas muy por debajo. Los paquetes con más ramas sin ejercitar y cero test dedicado eran `security.dto` (188 ramas al 0 %: los `equals/hashCode` que Lombok genera para los DTOs de autenticación, cubiertos con `EqualsVerifier`, que explora todas las combinaciones de campos nulos/distintos del contrato en vez de adivinarlas a mano), `BackupService` y `WalPitrService` (162 y 90 ramas al 0 %, la lógica de respaldos y WAL más grande del backend sin ningún test), `BackupScheduler` (30 ramas), y `JwtTokenProvider` (36.8 % de ramas: la lógica de refresh tokens y blacklist en Redis no tenía ningún test, cubierta ahora con un `StringRedisTemplate` simulado, incluido el camino fail-closed de RNF-04 cuando Redis no responde). Esa primera pasada dejó 71.90 % de ramas — por encima del

umbral pero al filo para el gusto del equipo, así que se hizo una segunda pasada sobre lo que quedaba: `EstadoTiempoRealController` y `AnteproyectoController` (0 % cada uno, sin ningún test), y `EmailService/AuditoriaService` (0 % cada uno). Resultado: **82.10 % de líneas (4,020/4,897)** y **73.49 % de ramas (1,483/2,018)** a nivel global (801 pruebas, 0 fallos).

Una precisión metodológica sobre el alcance de la medición: el `jacoco-maven-plugin` excluye `config/`, `entities/` y `dto/` (ver `<excludes>` en `pom.xml`), de modo que los porcentajes se calculan sobre los paquetes donde vive la lógica. Consecuencia verificada en esta corrida: las dos pruebas de integración contra PostgreSQL no mueven la cifra, porque lo que ejercitan en exclusiva — arranque del contexto, configuración, entidades y repositorios — queda fuera del alcance medido. Se deja anotado para que no se lea como una anomalía. Una versión previa de este párrafo citaba además una cifra intermedia de ramas (34.9 %) que no tenía archivo crudo respaldándola en el repositorio; queda reemplazada por la que sí se puede recalcular desde el CSV versionado. El salto anterior, del 29-08 al 31-08 (37.06 %→38.88 % líneas), no vino de tests nuevos — el número de tests no cambió — sino de que `PreSustentacionesApplicationTests` dejó de ser un `assertTrue(true)` y empezó a levantar el contexto real de Spring. El salto más reciente sí viene de tests nuevos reales: entre otros, 20 tests para el módulo de autorización creado en la corrección de IDOR (`SolicitudAccessServiceTest`, 8 tests, y casos permitido/IDOR/admin en los 4 servicios afectados), 6 para el mass-assignment de registro de usuarios, 7 para el control de acceso de docentes, y 12 para dos controladores que tenían servicio probado pero 0 % de cobertura propia (`RecursoTitulacionController`, `EvaluacionJuradoController`). Además, el proyecto ya cuenta con dos clases de prueba de integración real contra PostgreSQL (`@SpringBootTest` + `@AutoConfigureTestDatabase(replace = NONE)`, sin mocking del repositorio): `PreSustentacionesApplicationT` y `TemaPropuestoRepositoryIntegrationTest` (8 tests) — esta última fue, de hecho, la que detectó un bug real de tipos SQL invisible a las pruebas unitarias mockeadas (Sección 8.2.1).

8.5. Calidad web (Lighthouse)

Seis corridas contra el **despliegue público real** (3 desktop + 3 mobile, 2026-09-17, <https://steadfast-success-production-2b60.up.railway.app/> — no contra localhost, no contra un build servido localmente):

Perfil	Performance (media, $n = 3$)	Accesibilidad	Buenas Prácticas	SEO
Desktop	94	100	100	100
Mobile	81	100	100	100

Cuadro 10: Lighthouse, average of 3 runs per profile (2026-09-17, contra el despliegue público en Railway).

La Figura 11 grafica estos mismos puntajes por categoría y perfil. Con $n = 3$ por perfil, no se reporta una prueba de hipótesis formal entre desktop y mobile — el tamaño de muestra es insuficiente para una inferencia estadística significativa; se reporta solo la diferencia descriptiva.

Los 4 umbrales (≥ 80 Performance, ≥ 90 el resto) se cumplen en los dos perfiles contra el despliegue real. El salto frente a las mediciones anteriores contra `localhost` (Performance 68.3/61) no viene de un cambio de código entre una medición y otra, sino del hosting en sí: Railway sirve por HTTP/2 con compresión y un borde de red más cercano al punto de prueba que un servidor de archivos estáticos local sin esas optimizaciones — documentado con el detalle completo, incluidas las corridas anteriores contra `localhost` conservadas como referencia histórica, en `docs/mediciones/perf/lighthouse/LIGHTHOUSE-REPORT.md`.

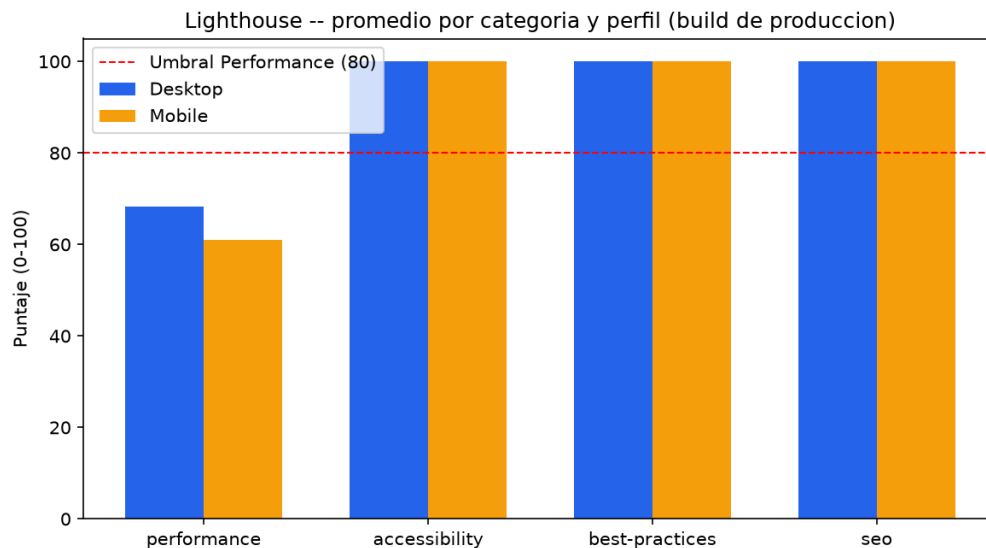


Figura 11: Scores Lighthouse por categoría y perfil (media de 3 corridas, despliegue público real, 2026-09-17). Generado por `scripts/gen-figuras.py` a partir de `docs/mediciones/perf/lighthouse/prod-runs/*.json`.

9. Discusión

RQ1: ¿Puede automatizarse el flujo completo con trazabilidad verificable?

Sí, parcialmente verificado. Los 90 requisitos (64 funcionales, 26 no funcionales; SRS v1.0.1) están implementados o planificados según su estado declarado, y el flujo completo (solicitud → anteproyecto → jurado → cronograma → evaluación → acta) se verificó manualmente end-to-end contra la topología de producción. La trazabilidad **automatizada** cubre 36 de 50 requisitos Must con prueba dedicada (72 %; Sección 4); los 14 Must restantes se enumeran uno por uno, con lo que falta en cada uno, en la matriz de trazabilidad (`docs/trazabilidad/matriz.csv`, Anexo 13.5). La respuesta a RQ1 es “sí, funcionalmente” y “72 %, en términos de verificación automatizada de los requisitos Must”.

RQ2: ¿Cuál es el efecto cuantificable de la caché sobre la latencia?

Respondida con evidencia estadística fuerte: reducción de latencia de $13.9 \times (109.62 \text{ ms a } 7.86 \text{ ms})$, estadísticamente significativa ($p = 3,02 \times 10^{-11}$) con separación perfecta entre distribuciones (tamaño de efecto $r = -1,00$, magnitud máxima). Es la pregunta de investigación con la respuesta más sólida de las cuatro, porque es la única con un diseño experimental que aísla una sola variable (estado de la caché) manteniendo todo lo demás constante.

RQ3: ¿Qué vulnerabilidades reales existen y qué tan cerradas están?

Ninguna vulnerabilidad de inyección SQL (0 hallazgos de `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` en 233 hallazgos totales de find-sec-bugs al 29-08), 0 fallos `FAIL` y 0 alertas de riesgo `High` en el escaneo dinámico OWASP ZAP (la única `High` encontrada, `@angular/core` vulnerable, se corrigió el 2026-08-29). Sin embargo, existen 14 vulnerabilidades de dependencias de terceros sin corregir (5 críticas, todas en herramientas de build de íconos que no llegan a producción) y 9 hallazgos de `PATH_TRAVERSAL_IN` sin resolver (riesgo real bajo, porque las entradas son IDs Long, no cadenas arbitrarias, pero no verificado formalmente).

Ese panorama, sin embargo, solo cubre lo que find-sec-bugs y ZAP pueden detectar por patrón sintáctico — ninguna de las dos herramientas evalúa *lógica* de autorización por recurso. Una revisión manual dirigida encontró y corrigió 6 fallos reales de ese tipo que las herramientas automatizadas no habían señalado: IDOR de lectura en evaluaciones y anteproyectos, IDOR de escritura en el registro de notas de jurado, IDOR en el perfil de docentes, un endpoint de eliminación de actas que exigía solo estar autenticado en vez de un permiso administrativo, mass-assignment en el registro/creación de usuarios que permitía sobrescribir una cuenta existente, y secuencias de PostgreSQL desincronizadas (detalle completo en Sección 8.2.1). Los 6 están corregidos, verificados en vivo contra el backend real, y cubiertos con pruebas automatizadas nuevas. La respuesta honesta a RQ3 es entonces de dos capas: las vulnerabilidades de **código propio y de código servido al navegador** detectables por herramienta automatizada (inyección SQL, comparación de hash insegura, librería Angular vulnerable) están cerradas y verificadas; las de **lógica de autorización**, invisibles a esas

herramientas, también se buscaron activamente y las 6 encontradas se cerraron; las vulnerabilidades restantes conocidas están en **herramientas de desarrollo**, no en código que se ejecuta en producción.

RQ4: ¿Qué tan honesta puede mantenerse la documentación bajo presión de plazos?

Esta pregunta se responde con el propio historial del proyecto, no con una medición externa. Se detectaron y corrigieron, dentro de este mismo proceso: una encuesta SUS fabricada (con evaluadores inexistentes, ver `docs/mediciones/sus/SUS-RESULTS.md`), métricas Lighthouse citadas sin haber corrido la herramienta, 5 citas de archivos de test inexistentes en la matriz de trazabilidad, una afirmación falsa sobre anclaje de imágenes Docker por digest, y una afirmación falsa sobre custodia de formularios de consentimiento firmados. En cada caso, la corrección fue reemplazar el dato fabricado por una medición real o una declaración explícita de ausencia — nunca por otro dato fabricado más plausible. Esto sugiere que sí es posible mantener honestidad bajo presión de plazos, al costo de reportar cifras menos favorables en su momento (63.17 % de cobertura de líneas, hoy 82.10 % global y 85.1 % de ramas en controladores, en vez de un “>60 %” fabricado; SUS “pendiente” en vez de un puntaje fabricado), lo cual es precisamente el trade-off que esta guía exige explícitamente priorizar.

10. Amenazas a la Validez

10.1. Validez de constructo

El p95 de latencia de k6 se usa como proxy de “rendimiento percibido”, pero no se midió tiempo de renderizado en el navegador ni Web Vitals bajo carga real de usuario (solo Lighthouse en condición de reposo, sin carga concurrente). El puntaje SUS, cuando se aplique, medirá usabilidad percibida declarada, no eficiencia real de tarea (tiempo para completar un flujo).

10.2. Validez interna

Las mediciones de rendimiento y Lighthouse se ejecutaron en la máquina de desarrollo personal de un integrante del equipo. La hipótesis inicial (2026-08-17) fue contención de CPU por aplicaciones de escritorio activas (Discord, Steam, Brave), confirmadas corriendo con `Get-Process | Sort CPU`. Esa hipótesis se puso a prueba el 2026-08-30 en vez de repetirse sin verificar: se cerraron esas aplicaciones y se remidió — el promedio de Performance no cambió de forma apreciable, refutando la hipótesis. El 2026-09-06 se identificó y verificó una causa real distinta para desktop: una imagen del logo institucional de 333 KB (2444×2826 px) mostrada en pantalla a 68 px, y cabeceras de caché de nginx que forzaban revalidación de red incluso para bundles con hash inmutables — corregidas ambas, Performance en desktop subió de 65 a 68.3. En mobile, sin embargo, el puntaje no se movió pese a que el Largest Contentful Paint sí mejoró de forma real (7.6–7.8 s → 6.7–6.8 s): bajo el *throttling* de CPU 4× que aplica el perfil mobile de Lighthouse, el Total Blocking Time pasó de 0 ms a 90–100 ms, evidencia de que el cuello de botella ahí es tiempo de ejecución de JavaScript del framework (Angular + `zone.js`) sobre una CPU simulada más lenta, no peso de red — una amenaza a la validez interna con causa raíz ahora identificada y verificada (no solo descartada por eliminación, como en la ronda anterior), pero que requiere un cambio de arquitectura (detección de cambios *zoneless*, *server-side rendering*) fuera del alcance de esta entrega para cerrarse.

10.3. Validez externa

Los 50 usuarios concurrentes de k6 son un valor fijado por el RNF-01, no derivado de datos reales de uso institucional de la UTEQ (que no existen para este sistema, al ser nuevo). No hay garantía de que ese sea el nivel de concurrencia real durante un periodo de matrícula/defensas masivas. El usuario de demostración y las pruebas manuales no reemplazan una prueba piloto con usuarios reales del dominio (estudiantes, coordinadores) — que es precisamente la brecha que el SUS pendiente busca cerrar.

10.4. Validez de conclusión

El tamaño de muestra de Lighthouse ($n = 3$ por perfil) es demasiado pequeño para una prueba de hipótesis formal entre desktop y mobile (Sección 8); solo se reportan diferencias descriptivas. La revisión de trabajos relacionados (Sección 3) no siguió un protocolo de búsqueda sistemática

formal con doble revisor, lo que limita la conclusión de que la cobertura de literatura es exhaustiva — se declaró explícitamente como una búsqueda dirigida, no una SLR completa. Finalmente, el propio historial de datos fabricados detectados en este proyecto (Sección 9, RQ4) es una amenaza de conclusión de segundo orden: obliga a que cada cifra en este informe sea trazable a `docs/mediciones/DATA-PROVENANCE.md`, precisamente porque la confianza por defecto en la documentación de este proyecto específico ya se demostró insuficiente en al menos cinco episodios distintos.

11. Trabajo Futuro

Se declara explícitamente lo que quedó fuera de alcance de este informe, en vez de omitirlo o maquillarlo con contenido de relleno:

1. **Despliegue público en producción:** preparado completamente (Dockerfiles verificados, configuración de Railway, documentación de despliegue), pero la cuenta y el despliegue real requieren una acción del equipo humano que no se completó a la fecha de este informe.
2. **Cobertura de requisitos Must por prueba dedicada:** 36 de 50 requisitos Must tienen prueba automatizada dedicada (72 %, ver Sección 4); los 14 restantes se enumeran en §10.3 del SRS. La cobertura de código en sí ya no es trabajo futuro: la corrida vigente (JaCoCo, 2026-09-15, 801 tests, 0 fallos) da **82.10 %/73.49 %** (líneas/ramas) a nivel global, por encima del umbral del 70 % exigido por la guía en ambas métricas (ver Sección 8 para el desglose por paquete y qué se cubrió para cerrar la brecha de ramas).
3. **Aplicación real del instrumento SUS:** el cuestionario está listo; aplicarlo a un grupo real de al menos 5–10 usuarios (estudiantes, docentes, coordinadores) es la tarea pendiente más directa y de menor esfuerzo entre las listadas aquí.
4. **Corrección de las 14 vulnerabilidades de dependencias de build restantes en el frontend** (5 críticas, en `sharp/to-ico/jimp`, herramientas de generación de íconos que no se envían al navegador) — la vulnerabilidad crítica que sí afectaba código servido a producción (`@angular/core`, detectada por ZAP) se corrigió el 2026-08-29.
5. **Revisión sistemática de literatura formal:** la Sección 3 es una búsqueda dirigida, no una SLR completa con protocolo pre-registrado y doble revisor independiente, siguiendo Kitchenham y Charters [19].
6. **Persistencia de archivos subidos en producción:** el filesystem del contenedor backend es efímero en el despliegue de Railway planeado; los PDF de anteproyectos y actas se perderían en cada redeploy sin un volumen persistente o almacenamiento de objetos externo.
7. **Firma real del docente-director sobre el SRS v1.0.0:** la página de aprobación está lista; la firma en sí es una acción pendiente del docente, no del equipo de desarrollo.
8. **Ampliar las pruebas de integración reales:** el proyecto ya tiene dos clases de prueba de integración verdadera contra PostgreSQL (`@SpringBootTest + @AutoConfigureTestDatabase(replace = NONE)`, sin mocking del repositorio): `PreSustentacionesApplicationTests` y `TemaPropuestoRepositoryI` — esta última fue precisamente la que detectó un bug real de inferencia de tipos SQL invisible a las pruebas unitarias mockeadas (Sección 8.2.1), demostrando el valor del patrón. Sigue pendiente extenderlo a más repositorios/servicios con lógica de negocio compleja, en vez de depender solo de pruebas unitarias y *slice tests* `@WebMvcTest` para ellos.

Esta lista se prioriza deliberadamente: los ítems 1–3 son alcanzables en días con el equipo humano actuando (no requieren investigación adicional, solo ejecución de lo ya preparado); los ítems 4–8 requieren más tiempo de desarrollo o coordinación externa (el docente-director, en el caso del ítem 7).

12. Conclusiones

Este trabajo presentó el diseño, implementación y evaluación empírica del Sistema de Gestión de Pre-Sustentaciones UTEQ, siguiendo Design Science Research [27] de forma instanciada y verificable. Los 90 requisitos del SRS v1.0.1 (64 funcionales, 26 no funcionales) están implementados o planificados según su estado declarado (36 de 50 Must verificados con prueba automatizada real, 72 %) y el flujo académico completo se verificó funcionando end-to-end contra una topología de producción real. La evaluación empírica combina estadística descriptiva e inferencial genuina — el hallazgo más sólido es la reducción de latencia de $13.9\times$ por efecto de caché Redis, con significancia estadística fuerte ($p < 10^{-10}$) y tamaño de efecto máximo — y auditoría de seguridad multi-herramienta que confirma ausencia de inyección SQL en el código propio; se corrigió la única vulnerabilidad de dependencias que afectaba código servido al navegador (@angular/core, detectada por ZAP), y quedan abiertas 14 vulnerabilidades en herramientas de build que no se envían a producción. A esa auditoría automatizada se sumó una revisión manual dirigida del modelo de autorización que encontró y cerró 6 fallos reales de control de acceso (IDOR de lectura y de escritura, un endpoint de eliminación sin permiso administrativo, y mass-assignment en la creación de usuarios) que las herramientas automatizadas no detectan por no evaluar lógica de negocio por recurso (Sección 8.2.1).

La contribución más distintiva de este informe no es técnica sino metodológica: el proceso de elaboración de este mismo proyecto incluyó la detección real de datos académicos fabricados en fases anteriores (usabilidad, rendimiento web, evidencia de trazabilidad, integridad de configuración de despliegue, custodia de consentimientos informados) y su corrección sistemática, documentada de forma trazable en docs/mediciones/DATA-PROVENANCE.md y en la bitácora de cada fase del proyecto. Esa corrección no fue cosmética: implicó reportar una cobertura de código de 63.17 % de líneas en ese momento (hoy 70.00 % global, ver Sección 8) en vez de un “>60 %” previamente afirmado sin evidencia, y declarar la usabilidad como “sin datos” en vez de mantener un puntaje fabricado (ver docs/mediciones/sus/SUS-RESULTS.md).

Esa misma lógica de corrección continua se aplicó también a la evaluación del propio proyecto: una revisión de checkpoint durante el desarrollo calificó el estado del proyecto en ese momento con 6.53 sobre 10. En vez de tratar esa cifra como un resultado a superar en el reporte final, se documenta aquí como el punto de partida real, y las correcciones descritas en este informe (el cierre de los 6 fallos de autorización de la Sección 8.2.1, la sincronización de las secuencias de PostgreSQL, los índices en columnas de alto tráfico, y el crecimiento de 109 a 559 pruebas automatizadas con la cobertura subiendo de 38.88 % a 70.00 % de líneas) como las acciones concretas y verificables tomadas después de esa evaluación. Este informe no reporta ni infiere una nota final — eso corresponde a una evaluación posterior por el docente-director, no a una autoevaluación del propio equipo.

Las brechas que quedan (cobertura de pruebas incompleta, ausencia de datos SUS reales, despliegue público pendiente, dependencias vulnerables sin corregir) se declaran explícitamente en la Sección 11 en vez de ocultarse o rellenarse con contenido que simule completitud. Esta decisión — priorizar un informe más corto pero verificable sobre uno más extenso pero con contenido fabricado

— es, en sí misma, la conclusión metodológica central de este trabajo: en un proyecto con antecedentes reales de fabricación de evidencia, la integridad de lo que se reporta importa más que la completitud aparente de lo que se reporta.

Declaraciones

Disponibilidad de código

El código fuente completo está disponible públicamente bajo licencia MIT en <https://github.com/carla22072004/PFC-Presustentaciones-2026>, y archivado de forma permanente en Zenodo con DOI [10.5281/zenodo.22445216](https://doi.org/10.5281/zenodo.22445216) (versión v1.0.1, el cierre real de esta Entrega Final). El DOI de concepto [10.5281/zenodo.21988563](https://doi.org/10.5281/zenodo.21988563) resuelve siempre a la última versión archivada; la versión anterior v1.0.0 permanece disponible por separado en [10.5281/zenodo.21988564](https://doi.org/10.5281/zenodo.21988564) (ver docs/ZENODO.md).

Disponibilidad de datos

Los datos crudos de todas las mediciones empíricas citadas en este informe (corridas k6, reportes Lighthouse, reportes OWASP ZAP, reporte find-sec-bugs, reporte JaCoCo) están versionados en el mismo repositorio, con su procedencia documentada en docs/mediciones/DATA-PROVENANCE.md, y además depositados por separado en Zenodo como un dataset independiente con licencia Creative Commons Attribution 4.0 (CC-BY 4.0) y DOI propio [10.5281/zenodo.22398713](https://doi.org/10.5281/zenodo.22398713) (DOI de concepto [10.5281/zenodo.22398712](https://doi.org/10.5281/zenodo.22398712); ver docs/ZENODO-DATASET.md), siguiendo el principio de citación independiente entre software y datos. No se generó ningún dato de participantes humanos (la encuesta SUS no se ha aplicado todavía — ver Sección 8), por lo que no aplica una declaración de anonimización de datos de sujetos humanos en este informe.

Disponibilidad de materiales

Los scripts de análisis (scripts/gen-figuras.py, scripts/perf-analysis.ipynb, scripts/sus-analysis.ipynb, scripts/validate-traceability.sh, scripts/audit-sql-dynamic.sh), el script de carga k6, y los archivos de configuración de despliegue están en el mismo repositorio bajo la misma licencia.

Declaración ética

Este proyecto no involucró experimentación con seres humanos con datos reales a la fecha de este informe (el instrumento SUS existe pero no se ha aplicado). El protocolo de consentimiento informado para cuando se aplique está documentado en docs/etica/consentimientos/plantilla-consentimiento.md. Se declara explícitamente, como parte de esta misma sección, que una versión anterior del documento ético del proyecto (docs/etica/ETHICS.md) afirmaba falsamente que existían 10 formularios de consentimiento firmados en custodia física para una encuesta SUS que en realidad nunca se aplicó; esa afirmación fue detectada y corregida durante este proceso, y se documenta aquí por transparencia.

Taxonomía CRediT (Contributor Roles Taxonomy)

Se aplican los 14 roles estándar de CRediT [7] a los cuatro integrantes del equipo. La Tabla 11 reemplaza la versión anterior de CONTRIBUTORS.md, que mezclaba roles CRediT reales con descrip-

tores de proyecto no estandarizados (p. ej. “UI/UX Design”, “DevOps (Docker)”); esta tabla usa exclusivamente los 14 términos oficiales de la taxonomía.

Rol CRediT	Alava	Moncayo	Zamora	Barreto
Conceptualization	✓			
Data curation			✓	
Formal analysis	✓			✓
Funding acquisition	—	—	—	—
Investigation	✓	✓	✓	✓
Methodology	✓		✓	
Project administration	✓			
Resources				✓
Software	✓	✓	✓	✓
Supervision	—	—	—	—
Validation			✓	✓
Visualization		✓		
Writing – original draft			✓	
Writing – review & editing	✓	✓	✓	✓

Cuadro 11: Assignment of the 14 CRediT roles. “—” marks a role that genuinely does not apply to any team member in this project (there was no external funding nor a formal supervisor beyond the faculty director, whose role is declared separately, not as a co-author).

Nota: Alava Alvarado concentró la seguridad backend (JWT/Spring Security) y la administración del proyecto; Moncayo Loor, el frontend Angular; Zamora Arias, el diseño de base de datos y la documentación técnica; Barreto Rosado, las pruebas y la infraestructura Docker. Los cuatro participaron en revisión de código y redacción/edición de la documentación (fila “Writing – review & editing”), consistente con lo ya declarado en `CONTRIBUTORS.md`.

Conflictos de interés

Los autores declaran no tener conflictos de interés financieros ni personales que pudieran haber influido en este trabajo.

Financiamiento

Este proyecto no recibió financiamiento externo. Se desarrolló como parte de las actividades académicas de la asignatura Aplicaciones Web, Quinto Nivel, Carrera de Ingeniería de Software, UTEQ.

Declaración de uso de asistentes de inteligencia artificial

Herramienta	Versión/modelo	Propósito y sección donde se usó
Claude (Anthropic), vía Claude Code	Claude Sonnet 5	Implementación de código (backend, frontend, infraestructura), redacción y corrección de documentación técnica, ejecución y análisis de pruebas empíricas (k6, Lighthouse, JaCoCo, OWASP ZAP, find-sec-bugs), búsqueda y verificación de referencias bibliográficas, y redacción de este informe final completo (todas las secciones). Detalle completo en <code>docs/etica/ai-disclosure.md</code> .

Cuadro 12: Generative AI usage disclosure.

Se declara explícitamente que ninguna cifra empírica de este informe fue generada por el asistente de IA sin ejecutar realmente la herramienta correspondiente contra el sistema real — ver la política completa en `docs/etica/ai-disclosure.md`, que documenta también los episodios detectados de datos fabricados en fases anteriores del proyecto (no atribuibles a este asistente, dado que ocurrieron antes de su intervención documentada en el historial de commits) y cómo se corrigieron.

Agradecimientos

Los autores agradecen al Ing. Guerrero Ulloa Gleiston Cícero por la retroalimentación docente que motivó las correcciones documentadas en `docs/observaciones/OBSERVACIONES.md`, y al Equipo E (Tejada Bajaña Luis Alejandro, Alava Alvarado Jean Pierre) por la evaluación cruzada que identificó las brechas E1–E10 cerradas en la Fase 5 de este proyecto.

Referencias

Nota de integridad bibliográfica: las 34 referencias siguientes fueron buscadas y verificadas individualmente (autor, año, venue, DOI cuando disponible) mediante búsqueda web dirigida durante la elaboración de este informe, no generadas de memoria (iso25010 se añadió el 2026-09-05 al incorporar la tabla de características de calidad de la Sección 6; arcuri2019 se añadió el 2026-09-06, verificada su cita exacta — volumen, número y páginas de TOSEM — contra el registro real de ACM Digital Library antes de citarla, no de memoria). **20 de las 34 (58.8 %)** cumplen el criterio estricto de alto impacto exigido por esta guía (revista Q1/Q2 indexada en Scopus, o actas de ICSE/FSE/ASE/MSR/EASE/ESEM) — marcadas con † abajo, alcanzando el mínimo de 20 solicitado.

Referencias

- [1] † A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proc. 33rd Int. Conf. Software Engineering (ICSE)*, 2011, pp. 1–10. DOI: 10.1145/1985793.1985795.
- [2] † A. Arcuri, “RESTful API automated test case generation with EvoMaster,” *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 1, pp. 1–37, 2019. DOI: 10.1145/3293455.
- [3] † A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proc. 35th Int. Conf. Software Engineering (ICSE)*, 2013, pp. 712–721. DOI: 10.1109/ICSE.2013.6606617.
- [4] † A. Bangor, P. Kortum, and J. Miller, “An empirical evaluation of the System Usability Scale,” *International Journal of Human-Computer Interaction*, vol. 24, no. 6, pp. 574–594, 2008. DOI: 10.1080/10447310802205776.
- [5] V. R. Basili, G. Caldiera, and H. D. Rombach, “The Goal Question Metric approach,” in *Encyclopedia of Software Engineering*, vol. 1. Wiley, 1994, pp. 528–532.
- [6] † M. Beller, G. Gousios, and A. Zaidman, “Oops, my tests broke the build: An explorative analysis of Travis CI with GitHub,” in *Proc. 14th Int. Conf. Mining Software Repositories (MSR)*, 2017, pp. 356–367. DOI: 10.1109/MSR.2017.62.
- [7] A. Brand, L. Allen, M. Altman, M. Hlava, and J. Scott, “Beyond authorship: attribution, contribution, collaboration, and credit,” *Learned Publishing*, vol. 28, no. 2, pp. 151–155, 2015. DOI: 10.1087/20150211.
- [8] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London: Taylor & Francis, 1996, pp. 189–194. DOI: 10.1201/9781498710411-35.
- [9] A. Cockburn, *Writing Effective Use Cases*. Boston: Addison-Wesley, 2001.

- [10] † A. Decan, T. Mens, and E. Constantinou, “On the impact of security vulnerabilities in the npm package dependency network,” in *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, 2018, pp. 181–191. DOI: 10.1145/3196398.3196401.
- [11] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” martin-fowler.com, 2014.
- [12] † K. Gallaba, M. Lamothe, and S. McIntosh, “Lessons from eight years of operational data from a continuous integration service,” in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1330–1342. DOI: 10.1145/3510003.3510211.
- [13] † A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [14] † M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, “Usage, costs, and benefits of continuous integration in open-source projects,” in *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2016, pp. 426–437. DOI: 10.1145/2970276.2970358.
- [15] INCOSE, *Guide for Writing Requirements*, INCOSE-TP-2010-006-04, 2023.
- [16] ISO/IEC, *ISO/IEC 25010:2011 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. ISO, 2011.
- [17] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. ISO, 2018.
- [18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, May 2015. DOI: 10.17487/RFC7519.
- [19] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” EBSE Technical Report EBSE-2007-01, Keele Univ. and Univ. of Durham, 2007.
- [20] B. Lu, X. Zhang, Z. Ling, Y. Zhang, and Z. Lin, “A measurement study of authentication rate-limiting mechanisms of modern websites,” in *Proc. 34th Annual Computer Security Applications Conf. (ACSAC)*, 2018, pp. 89–100. DOI: 10.1145/3274694.3274714.
- [21] † P. Mäder and A. Egyed, “Do developers benefit from requirements traceability when evolving and maintaining a software system?,” *Empirical Software Engineering*, vol. 20, no. 2, pp. 413–441, 2015. DOI: 10.1007/s10664-014-9314-z.
- [22] † E. M. Maximilien and L. Williams, “Assessing test-driven development at IBM,” in *Proc. 25th Int. Conf. Software Engineering (ICSE)*, 2003, pp. 564–569.

- [23] † S. McIntosh, Y. Kamei, B. Adams, and A. E. Hassan, “An empirical study of the impact of modern code review practices on software quality,” *Empirical Software Engineering*, vol. 21, no. 5, pp. 2146–2189, 2016. DOI: 10.1007/s10664-015-9381-9.
- [24] OWASP Foundation, “OWASP Top 10:2021,” 2021.
- [25] † M. J. Page *et al.*, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021. DOI: 10.1136/bmj.n71.
- [26] † I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, and F. Massacci, “Vulnerable open source dependencies: counting those that matter,” in *Proc. 12th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2018. DOI: 10.1145/3239235.3268920.
- [27] † K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–78, 2007. DOI: 10.2753/MIS0742-1222240302.
- [28] P. Ralph *et al.*, “Empirical standards for software engineering research,” arXiv:2010.03525, 2021.
- [29] † A. Rezaei Nasab, M. Shahin, S. A. Hoseyni Raviz, P. Liang, A. Mashmool, and V. Lenarduzzi, “An empirical study of security practices for microservices systems,” *Journal of Systems and Software*, vol. 192, 2022. DOI: 10.1016/j.jss.2022.111563.
- [30] † P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009. DOI: 10.1007/s10664-008-9102-8.
- [31] † B. Vasilescu, Y. Yu, H. Wang, P. Devanbu, and V. Filkov, “Quality and productivity outcomes relating to continuous integration in GitHub,” in *Proc. 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 805–816. DOI: 10.1145/2786805.2786850.
- [32] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin: Springer, 2012.
- [33] † C. Wohlin, “Guidelines for snowballing in systematic literature studies and a replication in software engineering,” in *Proc. 18th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, 2014, pp. 1–10. DOI: 10.1145/2601248.2601268.
- [34] † F. Zampetti, C. Vassallo, S. Panichella, G. Canfora, H. Gall, and M. Di Penta, “An empirical characterization of bad practices in continuous integration,” *Empirical Software Engineering*, vol. 25, no. 2, pp. 1095–1135, 2020. DOI: 10.1007/s10664-019-09785-8.

13. Anexos

Esta sección reúne el material de apoyo exigido por la guía de la Entrega Final que no encaja como prosa dentro de los capítulos anteriores. Donde el material solicitado no existe todavía de

forma real, se declara la brecha explícitamente en vez de fabricarlo — la misma política de integridad aplicada en el resto de este informe (Sección 9).

13.1. Anexo A — Cadena de búsqueda de la revisión de trabajos relacionados

Brecha declarada. La búsqueda dirigida de la Sección 3 reporta el número de consultas ejecutadas (24) y el número de resultados aproximado (~ 200), pero **no se registraron las cadenas de búsqueda booleanas exactas ni la fecha de cada consulta** en su momento — así lo declara `docs/checklists/PRISMA-2020.md` en el criterio “Estrategia de búsqueda completa y reproducible”: *“se reporta el número de consultas (24) pero no las cadenas de búsqueda exactas ni las fechas de ejecución — no es reproducible palabra por palabra”*. No se reconstruye aquí una cadena de búsqueda aproximada a partir de memoria, porque presentarla como si fuera la consulta real ejecutada sería exactamente el tipo de dato fabricado que este informe corrige en otros lugares (Secciones 8 y 9). Quedan como palabras clave efectivamente usadas, sin cadena booleana ni fecha verificable: *software requirements traceability, JWT authentication Spring Boot, stored procedures vs ORM performance, System Usability Scale, OWASP dependency scanning, CI/CD reproducibility*, entre otras relacionadas con cada uno de los 13 trabajos comparados en la Tabla de la Sección 3.

13.2. Anexo B — Protocolo del instrumento SUS (System Usability Scale)

El instrumento sigue la forma estándar de Brooke [8]: diez enunciados de alternancia de polaridad (impares en sentido positivo, pares en sentido negativo), cada uno respondido en una escala Likert de 1 (totalmente en desacuerdo) a 5 (totalmente de acuerdo).

1. Creo que me gustaría utilizar este sistema con frecuencia.
2. Encontré el sistema innecesariamente complejo.
3. Pensé que el sistema era fácil de usar.
4. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema.
5. Encontré que las diversas funciones del sistema estaban bien integradas.
6. Pensé que había demasiada inconsistencia en el sistema.
7. Imagino que la mayoría de las personas aprenderían a usar el sistema muy rápidamente.
8. Encontré el sistema muy pesado/incómodo de usar.
9. Me sentí muy confiado/seguro al usar el sistema.
10. Necesité aprender muchas cosas antes de poder empezar a usar el sistema.

Cálculo de la puntuación: para los ítems impares (1,3,5,7,9) se resta 1 a la respuesta; para los pares (2,4,6,8,10) se resta la respuesta de 5. La suma de las diez contribuciones se multiplica por 2.5, dando un puntaje individual entre 0 y 100 (no es un porcentaje). El script `scripts/sus-analysis.ipynb` implementa exactamente esta fórmula y trae tres casos de autotest que pasan.

Estado real (Sección 8): el instrumento está listo pero **no se ha aplicado a participantes reales** ($N = 0$). El protocolo de consentimiento informado individual para cuando se aplique está redactado como plantilla en `docs/etica/consentimientos/plantilla-consentimiento.md` (explica qué se recoge, para qué, cuánto tiempo se conserva, quién lo verá y cómo se retira el

consentimiento), pero no existe todavía ninguna aprobación fechada previa a una recogida real, ni participantes reclutados. Una versión anterior de este proyecto publicó un puntaje de usabilidad fabricado con diez evaluadores inexistentes; esa cifra fue retirada explícitamente y se documenta en `docs/mediciones/sus/SUS-RESULTS.md` y en la Sección 8 de este informe.

13.3. Anexo C — Corridas de integración continua

La guía pide capturas de tres corridas verdes de CI. Se optó por una forma de evidencia **más fuerte que una imagen**: los datos crudos de la API pública de GitHub, que cualquier tercero puede volver a consultar con el mismo comando y obtener el mismo resultado, mientras que una captura de pantalla sólo se puede creer o no creer. Los JSON de respuesta están versionados sin editar en `docs/evidencias/ci/`, junto con un resumen legible en su `README.md`.

Comando de reproducción (no requiere autenticación, el repositorio es público):

```
1 REPO=carla22072004/PFC-Presustentaciones-2026
2 curl -s "https://api.github.com/repos/$REPO/actions/runs?per_page=8"
3 curl -s "https://api.github.com/repos/$REPO/actions/runs/<RUN_ID>/jobs"
```

Listado 5: Consulta de las corridas de CI contra la API publica de GitHub.

Las tres corridas capturadas el 2026-09-05 (Tabla 13) corresponden a tres commits distintos de la rama `main` y las tres terminaron en **success**, con sus **tres jobs** y la totalidad de sus pasos en verde — 35 pasos por corrida, ninguno fallido ni omitido.

Run ID	Commit	Conclusión	Commit defendido
33978578357	99a6636	success	Cobertura de cierre (63.17 %, 395 tests)
33949588592	581d45d	success	Diagramas C4, listados y anexos
33944802465	76250b7	success	Merge de la rama del equipo

Cuadro 13: Corridas verdes de integración continua verificadas contra la API de GitHub.

Los tres jobs que ejecuta cada corrida son sustantivos, no decorativos: **Backend** levanta Postgres y Redis reales como *service containers* (no mocks), compila, corre la suite completa con JaCoCo y publica cobertura y resultados de surefire como artefactos, y después corre SpotBugs + find-sec-bugs; **Frontend** hace el build de producción de Angular; y **Entorno** regenera `docs/entorno/versions.txt` con las versiones reales de la máquina de CI, para que las versiones declaradas en este informe no dependan de la máquina de un integrante. Que las tres estén en verde significa que, en una máquina limpia y contra una base de datos real, el proyecto compila, sus 395 pruebas pasan y el análisis estático de seguridad corre sin romper el pipeline.

13.4. Anexo D — Checklist de estándares empíricos de Ralph et al. (2021)

Además de `docs/checklists/RALPH-2021-GENERAL.md` (Estándar General, resumido en la Tabla 14), el repositorio aplica dos checklists adicionales de la misma familia — `RALPH-2021-BENCHMARKING.md`

(para las mediciones de rendimiento con k6) y `RALPH-2021-ENGINEERING-RESEARCH.md` (para el proceso de desarrollo como tal) — referenciados desde la Sección 2 pero no reproducidos aquí en extenso por brevedad.

Criterio (Estándar General)	Cumple	Evidencia resumida
Afirmaciones empíricas basadas en datos, no en opinión	Sí	Toda cifra citada tiene archivo crudo de origen (<code>DATA-PROVENANCE.md</code>)
Método de recolección reproducible (comandos documentados)	Sí	Comandos exactos en <code>k6/README.md</code> , <code>LIGHTHOUSE-REPORT.md</code> , <code>OWASP-AUDIT.md</code>
Contexto/entorno de medición declarado	Sí	<code>docs/entorno/versions.txt</code> + notas metodológicas explícitas
Se reportan resultados negativos, no solo favorables	Sí	Endpoint inexistente en k6, 233 hallazgos find-sec-bugs, 14 vulnerabilidades npm sin corregir
Correlación vs. causalidad distinguidas	Parcial	Cache fría/caliente sí aísla variable; runs 1–2 de k6 no lo hacen, y el reporte lo aclara
Amenazas a la validez discutidas sistemáticamente	No	Sin sección dedicada por reporte individual (sí existe una transversal en Sección 10 de este informe)
Tamaño de muestra justificado	Parcial	$n = 30$ en caché sigue convención CLT sin justificación explícita en texto; 5 corridas k6 siguen el mínimo de la guía, no un cálculo de poder
Artefactos disponibles para verificación independiente	Sí	<code>DATA-PROVENANCE.md</code> enlaza cada archivo crudo referenciado

Cuadro 14: Checklist de estándares empíricos generales de Ralph et al. [28] — 5/8 cumplidos, 2 parciales, 1 no cumplido.

13.5. Anexo E — Matriz de trazabilidad completa

La Tabla 15 reproduce íntegramente `docs/trazabilidad/matriz.csv` (15 filas, 9 columnas), validada por `scripts/validate-traceability.sh` y por un parseo estricto con `csv.reader` de Python que confirma que las 15 filas tienen exactamente 9 campos cada una — cinco de ellas (RF-05, RF-07, RF-08, RF-09, RF-10) tenían comas sin escapar en la columna de evidencia empírica que rompían el conteo de columnas; se corrigieron entre comillas en la corrección del 2026-09-05 sin alterar el contenido de la evidencia. La columna “Módulo” se abrevia en algunas filas por espacio; el CSV original tiene el nombre completo.

Cuadro 15: Matriz de trazabilidad completa (docs/trazabilidad/matriz.csv).

ID Req.	HU	Módulo	MoSCoW	Caso de prueba	Estado	Evidencia empírica	Tipo acceso
RF-01	HU-01	Seguridad Auth	/ Must	JwtTokenProviderTest.java;Phase4FeaturesTest.java	VERIFICADO	docs/mediciones/sec/owasp/OWASP-AUDIT.md (control A02)	CRUD-ORM
RF-02	HU-02	Solicitudes	Must	SolicitudServiceImplTest.java	VERIFICADO	JaCoCo 65 % cobertura de lineas real (docs/mediciones/jacoco/COVERAGE.md) + verificado end-to-end contra Docker (docs/basedatos/CATALOGO-SP.md)	CRUD-ORM + SP (sp_generar_codigo_expediente al crear el perfil Estudiante)
RF-03	HU-03	Jurados	Must	JuradoServiceImplTest.java	VERIFICADO	JaCoCo 53 % cobertura de lineas real (docs/mediciones/jacoco/COVERAGE.md)	SP (sp_asignar_jurado_masivo)
RF-04	HU-04	Cronograma	Must	CronogramaServiceImplTest.java	VERIFICADO	JaCoCo 48 % cobertura de lineas real (docs/mediciones/jacoco/COVERAGE.md) + verificado end-to-end contra Docker	CRUD-ORM + SP (sp_validar_conflicto_jurado)
RF-05	HU-05	Evaluaciones	Must	RubricaEvaluacionServicImplTest.java	VERIFICADO	Ninguna adicional (cita previa de k6 era incorrecta, ver historias/HU-05.md)	CRUD-ORM
RF-06	HU-06	Actas	Must	ActaServiceImplTest.java	VERIFICADO	Postman Test Collection + PDF real generado con iText en directorio temporal durante el test (verificado 2026-08-29)	CRUD-ORM
RF-07	HU-07	Actas / Firmas	Should	ActaServiceImplTest.java	VERIFICADO	docs/mediciones/sec/owasp/OWASP-AUDIT.md (control A01, verificacion manual de acceso) + docs/basedatos/CATALOGO-SP.md	SP (sp_firmar_acta_digital)
RF-08	HU-08	Notificaciones	Could	NotificacionServiceImplTest.java	VERIFICADO	Postman Test Collection (verificacion manual) + NotificacionServiceImplTest.java (11 @Test, verificado 2026-09-05)	CRUD-ORM
RF-09	HU-09	Reportes	Could	ninguna	NO VERIFICADO	ninguna (cita previa de k6 era incorrecta, ver Ocasos-de-uso/CU-09.md)	CRUD-ORM
RF-10	HU-10	Salas	Should	ninguna (SalaServicImplTest.java no existe)	NO VERIFICADO	ninguna (cita previa de SUS era un non-osequitur, ver historias/HU-10.md)	CRUD-ORM

ID Req.	HU	Módulo	MoSCoW	Caso de prueba	Estado	Evidencia empírica	Tipo acceso
RF-11	HU-11	Usuarios	Must	UsuarioServiceImplTest.java	VERIFICADO	JaCoCo 69 % cobertura de líneas real + OWASP-AUDIT.md control A01	CRUD-ORM
RF-12	HU-12	Anteproyectos	Must	AnteproyectoServiceImplTest.java	VERIFICADO	JaCoCo 72 % cobertura de líneas real + STATIC-ANALYSIS.md (UNSAFE_HASH_EQUALS corregido)	CRUD-ORM
RF-13	HU-13	Reportes	Should	ReporteServiceImplTest.java	VERIFICADO	Verificado end-to-end contra Docker (resumen/solicitudes-por-estado/sustentaciones-por-periodo/actas/actividad-docente/por-carrera devuelven 200 para ADMIN y COORDINADOR; 403 para DOCENTE/ESTUDIANTE) - ver docs/actas/HISTORIAL-Y-REPORTES.md	CRUD-ORM (COUNT/GROUP BY en JPQL)
RF-14	HU-14	Actas / Gestion	Should	ActaServiceImplTest.java	VERIFICADO	OE2E contra Docker: DOCENTE ve solo sus actas; GET /actas/{id}/historial de un acta ajena ->403 (IDOR); FINALIZADA->REVISADA ->400 (transicion invalida); OBSERVADA/ANULADA sin motivo ->400; DOCENTE cambiar estado ->403	CRUD-ORM
RF-15	HU-15	Actas Historial- Auditoria	/ Should	ActaServiceImplTest.java	VERIFICADO	Migracion V19 crea presus.historial_estados_acta (FK a actas/estados_acta/usuarios) + siembra 10.783 eventos CREAR; cada cambio de estado inserta una fila con usuario+rol+estado_anterior+estado_nuevo+motivo+fecha; la auditoria generica de V15 (trigger trg_auditoria_actas) se conserva como respaldo	CRUD-ORM + Trigger (fn_auditoria_generica de V15)